

پروژه درس پردازش تصاویر دیجیتال آقای دکتر ابریشمی مقدم

Digital Image Processing

کسری عطار کاشانی

Kasra Attar Kashani

عنوان پروژه:

تحلیل تصاویر پزشکی با یادگیری عمیق با اتوانکودر نویزدا

Deep-Learning Autoencoder Denoising for Medical Image Diagnosis

موضوع اصلی پروژه طراحی و ارزیابی یک سامانه‌ی تشخیص خودکار کامپیوتری (Computer-Aided Diagnosis CAD) برای شناسایی بیماری‌های ریوی به‌طور خاص ذات‌الریه (Pneumonia) و کووید-۱۹ (COVID-19) از روی تصاویر پزشکی قفسه‌ی سینه است. این سامانه که مقاله آن را CADTra نامیده‌اند (ترکیب CAD و Transfer learning) روی دو نوع تصاویر پزشکی کار می‌کند:

- تصاویر اشعه‌ی ایکس (X-ray) قفسه‌ی سینه برای طبقه‌بندی سه‌کلاسه (کووید-۱۹ / ذات‌الریه / سالم)
- تصاویر سی‌تی‌اسکن (CT Scan) قفسه‌ی سینه برای طبقه‌بندی دوتایی (کووید-۱۹ / سالم).

Innovation اول مقاله تمرکز آن بر نقش «نویز» در تصاویر پزشکی و استفاده از یک اتوانکودر کانولوشنال نویزدا (Convolutional Denoising Autoencoder) به‌عنوان یک مرحله‌ی میانی میان پیش‌پردازش و طبقه‌بندی است.

ایده این بود که اگر قبل از ورود تصاویر به شبکه‌ی Classification، نویز آن‌ها کاهش یابد و ویژگی‌های مهم و ساختاری آن‌ها برجسته‌تر شود، دقت تشخیص مدل‌های طبقه‌بند افزایش خواهد یافت.

مقاله شامل آموزش و ارزیابی ۱۳ معماری مختلف، روی دو مجموعه داده، با سه نسبت تقسیم آموزش/آزمون، قبل و بعد از اعمال نویزدایی آزموده است.

انگیزه‌ی چندلایه مقاله:

- **تشخیص ذات‌الریه.** به‌ویژه در کشورهای جهان سوم به‌شدت پرهزینه است و تاخیر در تشخیص جان بیمار را تهدید کند. علائم ذات‌الریه با کووید-۱۹ شباهت دارد که تشخیص را دشوار می‌کند.
- در دوران کووید-۱۹ (PCR) استاندارد طلایی تشخیص بود، اما این آزمون گران و پیچیده است. در مقابل بررسی رادیولوژیک تصاویر قفسه‌ی سینه (X-ray و CT) سریع‌تر به بیماران و کاهش سرعت شیوع کمک می‌کند.
- **نیاز به تفسیر تخصصی.** یکی از مشکلات جدی در کار با تصاویر پزشکی، نیاز به رادیولوژیست برای تفسیر آن‌هاست. سامانه‌های تشخیص کامپیوتری به متخصصان کمک می‌کنند، چون به‌مراتب سریع‌تر و در بسیاری موارد دقیق‌تر عمل می‌کنند.
- **کمبود داده و شباهت کلاس‌ها.** به دلیل شباهت قابل توجه میان انواع ذات‌الریه و محدودیت تعداد تصاویر پزشکی برای بیماری‌های نوظهور مرتبط با ذات‌الریه (مانند کووید-۱۹ در آغاز همه‌گیری)، تشخیص خودکار این بیماری‌ها یک چالش جدی است. مقاله، یادگیری انتقالی (Transfer Learning) را راه حل امیدوارکننده معرفی می‌کند
- **نویز در بینایی ماشین.** کاهش نویز (Denoising) برای تحلیل درست تصاویر پزشکی ضروری است. مقاله استدلال می‌کند که روش‌های سنتی کاهش نویز در حالت خطی بودن ویژگی‌ها عملکردی مشابه اتوانکودر دارند، اما در حالت غیرخطی بودن ویژگی‌ها که در تصاویر پزشکی حاکم است روش‌های سنتی در بازسازی تصویر از نویز ناکام می‌مانند. اتوانکودرهای مبتنی بر CNN می‌توانند با بهره‌گیری از همبستگی‌های فضایی بالای پیکسل‌ها، نویز را کاهش دهند.

مقاله یک مدل تشخیص خودکار به نام CADTra را پیشنهاد می‌کند:

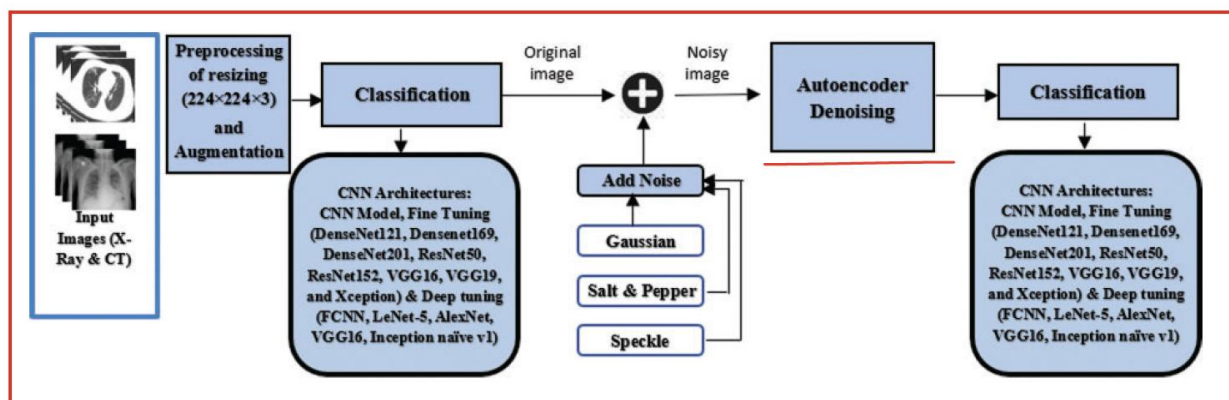
- 1- طبقه‌بندی (Classification)
- 2- اتوانکودر نویززا (Autoencoder Denoising)
- 3- یادگیری انتقالی (Transfer Learning)

اول پیش‌پردازش روی تصاویر پزشکی انجام می‌شود: خواندن مجموعه داده، افزایش داده (Augmentation) و یکسان‌سازی اندازه‌ی همه‌ی تصاویر به $224 \times 224 \times 3$ (طول، عرض و کانال رنگ) برای جلوگیری از بیش‌برازش (Overfitting) و آماده‌سازی ورودی CNN.

بعد اتوانکودر نویززا با یک تابع هزینه‌ی اصلاح‌شده، بر توزیع گاوسی برای خروجی رمزگشا (Decoder) آموزش داده می‌شود تا شانس بازیابی ورودی را بیشینه کند و ویژگی‌های تصویر را شفاف‌تر نمایش دهد. هدف، بهبود فرآیند تشخیص است.

سپس طبقه‌بندی با یک مدل یادگیری انتقالی و یک شبکه‌ی عصبی کانولوشنال چهارلایه (FCNN) برای تشخیص ذات‌الریه انجام می‌شود. طبقه‌بندی دوتایی روی تصاویر CT و طبقه‌بندی چندکلاسه روی تصاویر X-ray.

در انتها یک مطالعه‌ی مقایسه‌ای برای عملکرد طبقه‌بندی «با نویزدایی» و «بدون نویزدایی» ارائه می‌شود تا اثر واقعی مرحله‌ی AD کمی‌سازی شود.



اهداف این مقاله:

- تشخیص میان موارد ذات‌الریه، کووید-۱۹ و سالم با تکیه بر چند منبع تصویربرداری پزشکی (X-ray و CT)
- آموزش عمیق CNN برای مدل‌های یادگیری انتقالی و مدل FCNN بدون استفاده از CADTra (به‌عنوان مبنا)
- آموزش CADTra روی مجموعه داده‌های مختلف شامل افزودن نویز و سپس حذف آن با AD
- استفاده از CADTra برای بهبود همان آموزش عمیق و عملکرد طبقه‌بندی مدل‌های انتقال دانش
- استفاده از مدل‌های آماده برای مقایسه‌های دقیق.

مقاله گزارش می‌کند که مدل پیشنهادی به دقت حدود ۹۸٪ برای طبقه‌بندی دوتایی و ۹۹٪ برای طبقه‌بندی چندکلاسه دست یافته است و در مقایسه با مدل‌های روز CNN برتری نشان می‌دهد.

همچنین کیفیت نویززدایی با دو معیار PSNR (نسبت پیک سیگنال به نویز) و SSIM (شاخص شباهت ساختاری) سنجیده شده و سه نوع نویز گاوسی (Gaussian)، نمک و فلفل (Salt & Pepper) و لکه‌ای (Speckle) با واریانس‌های مختلف بررسی شده‌اند

نتیجه‌ی مقاله این بوده که نویز گاوسی شدیدترین نوع نویز برای این کاربرد است.

دستاوردها و نوآوری‌های ادعا شده در مقاله:

مقاله در چکیده و بخش نتیجه‌گیری خود، دستاوردهای زیر را برای CADTra برمی‌شمرد که در بازتولید این پروژه، همین دستاوردها محور سنجش و مقایسه قرار گرفته‌اند:

- دقت میانگین FCNN پیشنهادی روی تصاویر X-ray برابر $98/42\%$ و روی تصاویر CT برابر $98/34\%$ (پس از نویزدایی با AD) در حالی که بدون AD این اعداد $98/38\%$ و $97/64\%$ بوده‌اند. یعنی بهبودی کوچک اما مثبت از نویزدایی.
- بهبود عملکرد تقریباً همه‌ی مدل‌های یادگیری انتقالی پس از عبور تصاویر از مرحله‌ی AD. برای مثال ResNet152 با پیش‌آموزش از $98/11\%$ به $98/31\%$ روی X-ray و از $96/56\%$ به $97/79\%$ روی CT رسیده است.
- مقایسه‌ی جامع با مدل‌های (CapsNet، Stacked-Autoencoder، DCN) مقاله که برتری CADTra را با دقت $98/42\%$ (X-ray) و $98/34\%$ (CT) نشان می‌دهد.
- ارزیابی کامل اتوانکودر نویزدا روی سه نوع نویز و پنج سطح واریانس با معیارهای PSNR و SSIM که نشان می‌دهد PSNR خروجی روی X-ray در بهترین حالت (نمک و فلفل با چگالی 0.05)

پیشنهاد کار آینده که تو مقاله خودشون نوشته بودن:

استفاده از شبکه‌های مولد تخصصی (GAN) برای ابر-وضوح (Super-Resolution) پس از مرحله‌ی نویزدایی، به‌منظور بهبود بیشتر عملکرد طبقه‌بندی

معماری شبکه‌های مورد استفاده در مقاله

معماری اتوانکودر نویزدا، شبکه‌ی FCNN و مدل‌های یادگیری انتقالی به همراه ساختار کلی مدل ترکیبی CADTra و گام‌های ساخت آن:

1- ساختار کلی سه مرحله‌ای مدل CADTra:

مرحله‌ی اول: پیش‌پردازش (Pre-processing): خواندن و پردازش مجموعه داده و افزایش تصاویر (Augmentation) است. از آنجا که تصاویر از بیش از یک منبع جمع‌آوری شده‌اند و اندازه‌های متفاوتی دارند، این مرحله همه‌ی تصاویر را به اندازه‌ی ثابت $224 \times 224 \times 3$ تغییر اندازه می‌دهد. این اعداد به ترتیب کانال، عرض و طول تصویر هستند و یکسان‌سازی آن‌ها برای جلوگیری از بیش‌برازش و امکان طبقه‌بندی با معماری CNN ضروری است.

مرحله‌ی دوم: اتوانکودر نویزدا (Autoencoder Denoising): در این مرحله انواع مختلف نویز (گوسی، لکه‌ای و نمک و فلفل) پردازش می‌شوند تا اثرشان بر فرآیند طبقه‌بندی کاهش یابد. این مرحله اصل نوآوری مقاله است.

مرحله‌ی سوم: طبقه‌بندی (Classification): در این مرحله از مدل FCNN (آموزش کامل از صفر) و مدل‌های یادگیری انتقالی برای طبقه‌بندی نهایی تصاویر استفاده می‌شود.

2- شبکه‌ی اتوانکودر نویززا (Denoising Autoencoder):

اتوانکودر مقاله از دو بخش (Encoder) و (Decoder) تشکیل شده که هر کدام سه لایه دارند، به علاوه یک مرحله‌ی نرمال‌سازی دسته‌ای (Batch Normalization) در آغاز مدل. در مجموع سه لایه‌ی کانولوشن (Convolution) و سه لایه‌ی کانولوشن (Convolution-Transposed) در این اتوانکودر به کار رفته‌اند.

Encoder تصویر ورودی را فشرده می‌کند تا همه‌ی ویژگی‌های قوی را استخراج و ویژگی‌های ضعیف (که نویز عمدتاً در همین دسته قرار می‌گیرد) را حذف کند. بعد Decoder بر اساس ترکیب و ویژگی‌های استخراج‌شده، تصویر را بازسازی می‌کند. هدف کلی این اتوانکودر «نویززدایی از تصاویر» است و فرآیند آموزش با مقایسه‌ی تصویر بازسازی‌شده با تصویر اصلی و اصلاح وزن‌ها برای به دست آوردن مشابه‌ترین تصویر به نسخه‌ی اصلی انجام می‌شود.

به بیان مقاله، استفاده از تعداد لایه‌های کمتر و رسیدن به هزینه‌ی محاسباتی پایین‌تر، عملکرد فرآیند نویززدایی تصویر را بهبود می‌بخشد به همین دلیل یک شبکه‌ی اتوانکودر کانولوشنال هشت لایه طراحی شده است.

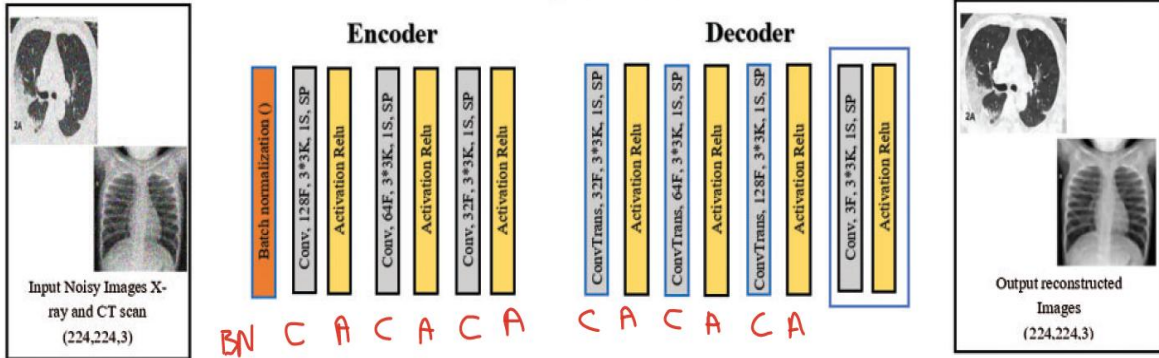
مشخصات دقیق معماری اتوانکودر مقاله به شرح زیر است:

- **ابعاد تصویر ورودی:** $3 \times 224 \times 224$ (تصویر رنگی).
- **Encoder:** یک لایه‌ی Batch Normalization در ابتدا و سپس سه لایه‌ی کانولوشن با فیلترهای ۱۲۸، ۶۴ و ۳۲ تایی به ترتیب برای لایه‌های ۱، ۲ و ۳. اندازه‌ی (Kernel) همه‌ی لایه‌ها 3×3 ، گام (Stride) برابر ۱ و (Padding) از نوع same است. بنابراین ابعاد مکانی تصویر در طول Encoder ثابت می‌ماند و فقط عمق (تعداد کانال) کاهش می‌یابد. خروجی Encoder دارای ابعاد $32 \times 224 \times 224$ است.
- **Decoder:** سه لایه‌ی ConvTrans با فیلترهای ۳۲، ۶۴ و ۱۲۸ تایی به ترتیب برای لایه‌های ۱، ۲ و ۳ (دقیقاً معکوس ترتیب رمزگذار) و در انتها یک لایه‌ی کانولوشن خروجی که تصویر نهایی $3 \times 224 \times 224$ را تولید می‌کند.

- تابع فعال‌سازی در تمام لایه‌های Conv و ConvTrans برابر ReLU (واحد اصلاح‌شده‌ی خطی) است

لایه	نوع	تعداد فیلتر / کانال	اندازه‌ی هسته	خروجی
۰	Batch Normalization	۳	—	$۲۲۴ \times ۲۲۴ \times ۳$
۱ (رمزگذار)	Convolution + ReLU	۱۲۸	۳×۳	$۲۲۴ \times ۲۲۴ \times ۱۲۸$
۲ (رمزگذار)	Convolution + ReLU	۶۴	۳×۳	$۲۲۴ \times ۲۲۴ \times ۶۴$
۳ (رمزگذار)	Convolution + ReLU	۳۲	۳×۳	$۲۲۴ \times ۲۲۴ \times ۳۲$ (نهفته)
۴ (رمزگشا)	ConvTranspose + ReLU	۳۲	۳×۳	$۲۲۴ \times ۲۲۴ \times ۳۲$
۵ (رمزگشا)	ConvTranspose + ReLU	۶۴	۳×۳	$۲۲۴ \times ۲۲۴ \times ۶۴$
۶ (رمزگشا)	ConvTranspose + ReLU	۱۲۸	۳×۳	$۲۲۴ \times ۲۲۴ \times ۱۲۸$
۷ (خروجی)	Convolution	۳	۳×۳	$۲۲۴ \times ۲۲۴ \times ۳$

Denoising Autoencoder



:Loss Function

نکته‌ی مهم دیگر در مقاله، تابع هزینه‌ی (Loss Function) اصلاح‌شده‌ی اتوانکودر است. در فاز آموزش، مدل برای کمینه‌کردن خطای بازسازی و بیشینه‌کردن شانس بازیابی ورودی آموزش داده می‌شود (چون هم Encoder هم Decoder غیرخطی هستند). آموزش با کمینه‌سازی تابع هزینه از طریق پس‌انتشار خطا (Backpropagation) انجام می‌شود تا قوی‌ترین ویژگی‌ها انتخاب شوند. مقاله ابتدا خطای میانگین مربعات (MSE) را تعریف می‌کند:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

که در آن y ورودی اصلی و $\hat{y}$ خروجی بازسازی‌شده است. سپس تابع هزینه‌ی اصلاح‌شده بر مبنای توزیع گاوسی برای خروجی رمزگشا معرفی می‌شود:

$$L(\theta) = - \sum_{i=1}^n \log p(y_i | \hat{y}_i)$$

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$Loss = -\log p(y_i | z) = -\log \prod_{i=1}^n N(y_i; \hat{y}_i, \sigma^2) = - \sum_{i=1}^n \log N(y_i; \hat{y}_i, \sigma^2) \propto \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

loss function

$$N(y_i; \hat{y}_i, \sigma^2)$$

یک توزیع گاوسی برای خروجی رمزگشا با واریانس است.

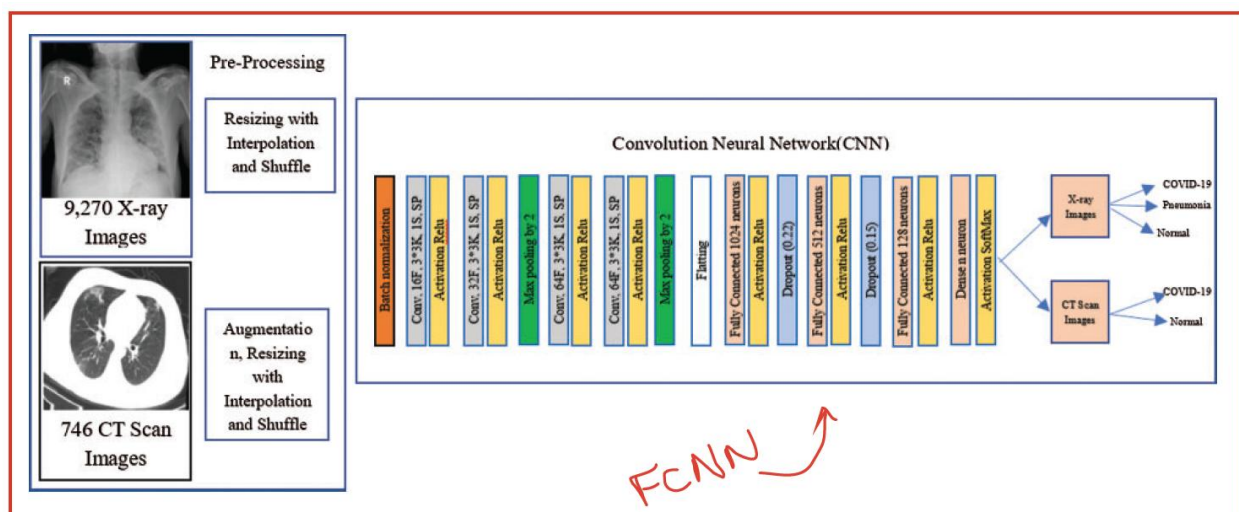
3- شبکه‌ی طبقه‌بند چهارلایه (FCNN)

برای آزمون مدل، مقاله یک FCNN (شبکه‌ی کانولوشنال چهارلایه با Full-Training) طراحی کرده است که نقش طبقه‌بند اصلی مقاله را دارد. چارچوب کلی CNN در مقاله شامل لایه‌های Batch Normalization، ورودی، کانولوشنال، تماماً متصل (Fully-Connected)، Pooling، Dropout، Dense و خروجی است و می‌توان آن را به صورت انتها به انتها (End-to-End) آموزش داد تا انتخاب و استخراج ویژگی و در نهایت پیش‌بینی یا طبقه‌بندی انجام شود.

مشخصات دقیق FCNN: ابتدا یک لایه‌ی Batch Normalization. بعد چهار لایه‌ی کانولوشن با ۱۶، ۳۲، ۶۴ و ۶۴ فیلتر به ترتیب، با تابع فعال‌سازی ReLU و اندازه‌ی هسته‌ی 3×3 ؛ همچنین دو لایه‌ی Max-Pooling با Pool Size برابر ۲؛ سپس سه لایه‌ی تماماً متصل (FC) با احتمالات Dropout برابر 0.22 و 0.15 و در نهایت استفاده از SoftMax به عنوان تابع فعال‌سازی خروجی و آنتروپی متقاطع (Cross-Entropy) به عنوان تابع هزینه در لایه‌ی طبقه‌بندی.

بلوک	محتوا	جزئیات
ورودی	تصویر	$224 \times 224 \times 3$
نرمال‌سازی	Batch Normalization	در ابتدای شبکه
کانولوشن ۱	Conv 3×3 + ReLU	۱۶ فیلتر
ادغام ۱	Max-Pooling	اندازه‌ی 2×2
کانولوشن ۲	Conv 3×3 + ReLU	۳۲ فیلتر
ادغام ۲	Max-Pooling	اندازه‌ی 2×2

بلوک	محتوا	جزئیات
کانولوشن ۳	Conv 3×3 + ReLU	۶۴ فیلتر
کانولوشن ۴	Conv 3×3 + ReLU	۶۴ فیلتر
تماماً متصل ۱	FC + ReLU + Dropout	احتمال Dropout = 0.22
تماماً متصل ۲	FC + ReLU + Dropout	احتمال Dropout = 0.15
تماماً متصل ۳	FC + Softmax	تعداد نورون = تعداد کلاس‌ها
تابع هزینه	Cross-Entropy	در لایه ی طبقه بندی



4-مدل‌های یادگیری انتقالی و آموزش کامل

مقاله علاوه بر FCNN از دو دسته‌ی بزرگ مدل برای طبقه‌بندی استفاده می‌کند که در مجموع ۱۳ معماری را پوشش می‌دهند:

دسته‌ی اول: آموزش کامل (Full-Training): چهار معماری AlexNet، LeNet-5، VGG16 و Inception (Naïve) در این حالت مدل از ابتدا روی داده‌ی پزشکی آموزش کامل می‌بیند.

دسته‌ی دوم: Pre-Training + Fine-Tuning: هشت معماری DenseNet169، DenseNet121، VGG19، VGG16، ResNet152، ResNet50، DenseNet201 و Xception. مقاله یادگیری انتقالی را به سه سناریوی اصلی تقسیم می‌کند:

- (Deep Tuning)
- (Shallow Tuning)
- (Fine Tuning)

و روش مورد استفاده‌ی خود را Fine Tuning معرفی می‌کند که هدفش آموزش لایه‌های بیشتر با تنظیم وزن‌های یادگیری است تا یک جهش معنادار در عملکرد به دست آید.

خلاصه‌ی ۱۳ معماری به کار رفته در مقاله در جدول زیر آمده است:

ردیف	مدل	روش آموزش در مقاله	اندازه‌ی ورودی
۱	FCNN (پیشنهادی)	آموزش کامل از صفر (Full-Training)	$224 \times 224 \times 3$
۲	LeNet-5	آموزش کامل (Full-Training)	$224 \times 224 \times 1$
۳	AlexNet	آموزش کامل (Full-Training)	$224 \times 224 \times 1$

ردیف	مدل	روش آموزش در مقاله	اندازه‌ی ورودی
۴	VGG16	آموزش کامل (Full-Training)	۲۲۴×۲۲۴×۱
۵	Inception Naïve V1	آموزش کامل (Full-Training)	۲۲۴×۲۲۴×۱
۶	DenseNet121	Pre-Training + Fine-Tuning	۲۲۴×۲۲۴×۳
۷	DenseNet169	Pre-Training + Fine-Tuning	۲۲۴×۲۲۴×۳
۸	DenseNet201	Pre-Training + Fine-Tuning	۲۲۴×۲۲۴×۳
۹	ResNet50	Pre-Training + Fine-Tuning	۲۲۴×۲۲۴×۳
۱۰	ResNet152	Pre-Training + Fine-Tuning	۲۲۴×۲۲۴×۳
۱۱	VGG16	Pre-Training + Fine-Tuning	۲۲۴×۲۲۴×۳
۱۲	VGG19	Pre-Training + Fine-Tuning	۲۲۴×۲۲۴×۳
۱۳	Xception	Pre-Training + Fine-Tuning	۲۹۹×۲۹۹×۳

► VGG16 (input 224x224x3)

=====		
=====		
Layer (type:depth-idx)	Output Shape	Param #
=====		
=====		
VGG	[1, 3]	--
└Sequential: 1-1	[1, 512, 7, 7]	--
└Conv2d: 2-1	[1, 64, 224, 224]	1,792
└ReLU: 2-2	[1, 64, 224, 224]	--
└Conv2d: 2-3	[1, 64, 224, 224]	36,928
└ReLU: 2-4	[1, 64, 224, 224]	--
└MaxPool2d: 2-5	[1, 64, 112, 112]	--
└Conv2d: 2-6	[1, 128, 112, 112]	73,856
└ReLU: 2-7	[1, 128, 112, 112]	--
└Conv2d: 2-8	[1, 128, 112, 112]	147,584
└ReLU: 2-9	[1, 128, 112, 112]	--
└MaxPool2d: 2-10	[1, 128, 56, 56]	--
└Conv2d: 2-11	[1, 256, 56, 56]	295,168
└ReLU: 2-12	[1, 256, 56, 56]	--
└Conv2d: 2-13	[1, 256, 56, 56]	590,080
└ReLU: 2-14	[1, 256, 56, 56]	--
└Conv2d: 2-15	[1, 256, 56, 56]	590,080
└ReLU: 2-16	[1, 256, 56, 56]	--
└MaxPool2d: 2-17	[1, 256, 28, 28]	--
└Conv2d: 2-18	[1, 512, 28, 28]	1,180,160
└ReLU: 2-19	[1, 512, 28, 28]	--
└Conv2d: 2-20	[1, 512, 28, 28]	2,359,808
└ReLU: 2-21	[1, 512, 28, 28]	--
└Conv2d: 2-22	[1, 512, 28, 28]	2,359,808
└ReLU: 2-23	[1, 512, 28, 28]	--
└MaxPool2d: 2-24	[1, 512, 14, 14]	--
└Conv2d: 2-25	[1, 512, 14, 14]	2,359,808
└ReLU: 2-26	[1, 512, 14, 14]	--
└Conv2d: 2-27	[1, 512, 14, 14]	2,359,808
└ReLU: 2-28	[1, 512, 14, 14]	--
└Conv2d: 2-29	[1, 512, 14, 14]	2,359,808
└ReLU: 2-30	[1, 512, 14, 14]	--
└MaxPool2d: 2-31	[1, 512, 7, 7]	--
└AdaptiveAvgPool2d: 1-2	[1, 512, 7, 7]	--
└Sequential: 1-3	[1, 3]	--
└Linear: 2-32	[1, 4096]	102,764,544
└ReLU: 2-33	[1, 4096]	--
└Dropout: 2-34	[1, 4096]	--
└Linear: 2-35	[1, 4096]	16,781,312

	└ReLU: 2-36	[1, 4096]	--
	└Dropout: 2-37	[1, 4096]	--
	└Sequential: 2-38	[1, 3]	1,049,603

=====

=====

Total params: 135,310,147

► FCNN (from scratch) (input 224x224x3)

=====

Layer (type:depth-idx)	Output Shape	Param #
PreviewFCNN	[1, 3]	--
└BatchNorm2d: 1-1	[1, 3, 224, 224]	6
└Conv2d: 1-2	[1, 16, 224, 224]	448
└Conv2d: 1-3	[1, 32, 112, 112]	4,640
└Conv2d: 1-4	[1, 64, 56, 56]	18,496
└Conv2d: 1-5	[1, 64, 56, 56]	36,928
└Linear: 1-6	[1, 128]	25,690,240
└Dropout: 1-7	[1, 128]	--
└Linear: 1-8	[1, 64]	8,256
└Dropout: 1-9	[1, 64]	--
└Linear: 1-10	[1, 3]	195

=====

=====

Total params: 25,759,209

Trainable params: 25,759,209

Non-trainable params: 0

Total mult-adds (Units.MEGABYTES): 280.19

=====

=====

Input size (MB): 0.60

Forward/backward pass size (MB): 14.05

Params size (MB): 103.04

Estimated Total Size (MB): 117.69

=====

=====

5- شبکه‌ی کلی و گام‌های هشت‌گانه‌ی ساخت CADTra

در مقاله FCNN در طبقه‌بندی دوتایی و سه کلاسه روی مجموعه داده‌های CT و X-ray به کار رفته‌اند. بعد شبکه‌ی اتوانکودر کانولوشنال برای استخراج ویژگی از تصاویر و بازسازی آن‌ها طراحی و به‌طور مستقل آموزش داده شده است. شبکه‌ی اتوانکودر قبل و بعد از مرحله‌ی نویزدایی به کار رفته‌اند تا اثر AD بر فرآیند طبقه‌بندی آزموده شود. در ادامه Fine-Tuning انجام می‌شود. طراحی کلی شبکه در مقاله در هشت گام است:

گام 1: ساخت مدل‌های یادگیری انتقالی و FCNN.

گام 2: استفاده از تابع هزینه‌ی (Cross-Entropy) برای آموزش مدل‌های یادگیری انتقالی و FCNN و ذخیره‌ی وزن‌های آموزش‌دیده.

گام 3: ساخت شبکه‌ی اتوانکودر نویزدا.

گام 4: استفاده از تابع هزینه‌ی میانگین مربعات خطا (MSE) برای آموزش AD و ذخیره‌ی وزن‌های آموزش‌دیده.

گام 5: استفاده‌ی مجدد از مدل‌های یادگیری انتقالی و FCNN پس از AD

گام 6: سازماندهی مجدد شبکه‌ی اتوانکودر نویزدا، مدل‌های یادگیری انتقالی و FCNN برای ساخت شبکه‌ی ترکیبی (CADTra).

گام 7: ثابت نگه‌داشتن (Freeze) وزن‌های شبکه‌ی اتوانکودر نویزدا در فرآیند نویزدایی تصویر؛ وزن‌های آموزش‌دیده ذخیره و به‌عنوان ورودی برای مدل‌های یادگیری انتقالی و FCNN استفاده می‌شوند.

گام 8: ریزتنظیم (Fine-Tuning) تمام پارامترهای شبکه‌ی کلی بر اساس وزن‌های اتوانکودر نویززا و یادگیری انتقالی؛ سپس ذخیره‌ی مدل نهایی پیشنهادی (CADTra).

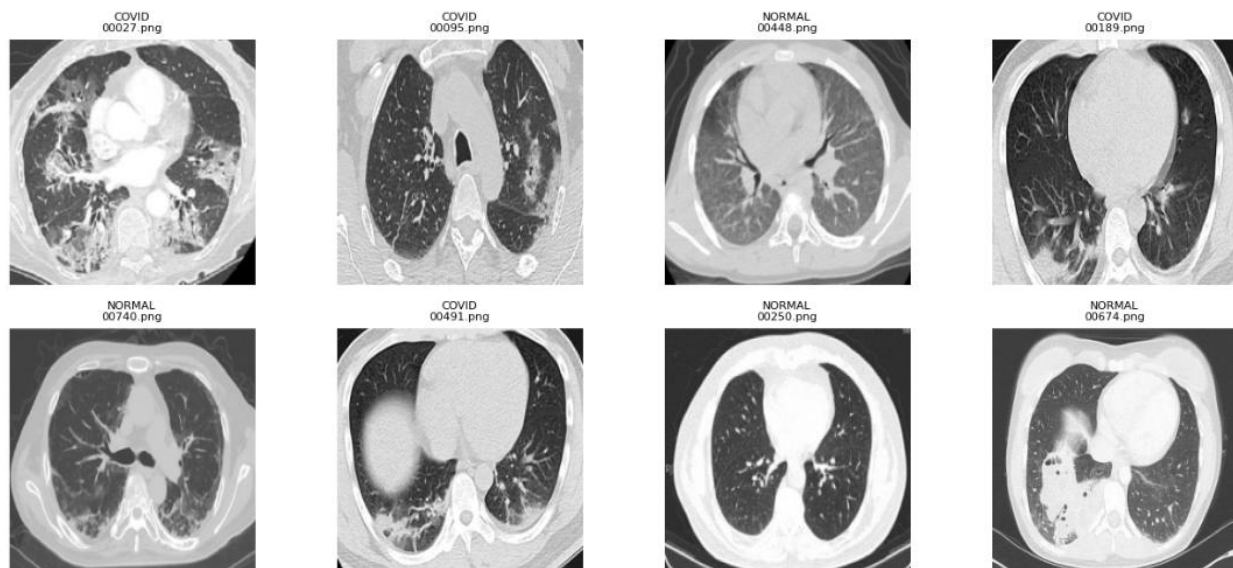
Database مورد استفاده

مقاله در مجموع ۹۲۰ تصویر X-ray و ۲۷۶۲ تصویر CT به دست آورده‌اند. این مجموعه داده‌ها از منابع عمومی و پلتفرم‌های مختلف گردآوری شده‌اند و چهار مرجع اصلی در مقاله برای آن‌ها ذکر شده است:

- مجموعه داده‌ی COVID-CT دانشگاه UCSD (گیت‌هاب UCSD-AI4H/COVID-CT): منبع تصاویر CT قفسه‌ی سینه که شامل دو دسته‌ی COVID و Non-COVID است.
- مجموعه داده‌ی گسترده‌ی تصاویر X-ray و CT قفسه‌ی سینه‌ی کووید-۱۹ (Extensive COVID-19 X-ray and CT Chest Images Dataset) گردآوری شده توسط Abd El-Samie و El-Shafai روی پلتفرم Mendeley Data (نسخه‌ی ۳).
- مجموعه داده‌ی معروف Chest X-Ray Pneumonia روی پلتفرم Kaggle (paultimothymooney/chest-xray-pneumonia): منبع تصاویر X-ray برای کلاس‌های سالم (NORMAL) و ذات‌الریه (PNEUMONIA).
- نسخه‌ی دیگری از مجموعه داده‌ی Mendeley (شناسه‌ی h65ywd2jr8) برای تکمیل تصاویر.

به این ترتیب، تصاویر X-ray دارای سه دسته هستند: ۱۱۶۱ تصویر کووید-۱۹، ۴۲۴۰ تصویر ذات‌الریه و ۳۸۰۰ تصویر سالم. تصاویر CT نیز دو دسته دارند: ۱۳۰۵ تصویر کووید-۱۹ و ۱۴۵۷ تصویر سالم که به صورت عمومی در دسترس‌اند.

سهم از کل (%)	تعداد تصاویر در مقاله	کلاس	Mode
12%	۱۱۶۱	کووید-۱۹ (COVID-19)	X-ray
46%	۴۲۴۰	ذات‌الریه (Pneumonia)	X-ray
42%	۳۸۰۰	سالم (Normal)	X-ray
	۹۲۰۱	جمع	X-ray
48%	۱۳۰۵	کووید-۱۹ (COVID-19)	CT
52%	۱۴۵۷	سالم (Normal)	CT
	۲۷۶۲	جمع	CT



مجموعه داده‌ی X-ray دارای تصاویری با اندازه‌های متفاوت است و از نظر تعداد نمونه در هر کلاس Imbalanced است. تصاویر کووید-۱۹ تنها ۱۲٪ از کل تصاویر را تشکیل می‌دهند، در حالی که ذات‌الریه ۴۶٪ و سالم ۴۲٪ سهم دارند. این می‌تواند به (Overfitting) منجر شود. برای این مشکل، مقاله لایه‌های Dropout را در مدل پیشنهادی به کار برده است.

داده‌های به‌کاررفته در شبیه‌سازی من و تفاوت حجمی آن با مقاله

در شبیه‌سازی این پروژه دقیقاً از همان دیتا مقاله استفاده شد

xray_normal: سه پوشه‌ی train/test/val از مجموعه‌ی chest_xray در مجموع ۳۱۶۶ تصویر.

xray_pneumonia: سه پوشه‌ی train/test/val از مجموعه‌ی chest_xray در مجموع ۸۵۴۶ تصویر.

xray_covid: پوشه‌ی COVID از مجموعه‌ی Mendeley ۸۰۸۸ تصویر.

ct_covid: پوشه‌ی CT_COVID ۶۹۸ تصویر.

ct_normal: پوشه‌ی CT_NonCOVID ۷۹۴ تصویر.

نکته در مورد دیتا:

تعداد تصاویر در برخی کلاس‌ها تصاویر بیشتر از مقاله و برخی کلاس‌ها تصاویر کمتری دارند (مهم‌تر از همه کلاس سالم X-ray با ۳۱۶۶ تصویر در برابر ۳۸۰۰، کلاس کووید CT با ۶۹۸ تصویر در برابر ۱۳۰۵ و کلاس سالم CT با ۷۹۴ تصویر در برابر ۱۴۵۷). علت این اختلاف شاید این باشد که نویسندگان مقاله از نسخه‌های خاص و

ترکیبی این مجموعه داده‌ها (و احتمالاً نسخه‌های حذف شده یا تکمیلی) استفاده کرده‌اند که امروز به صورت دقیق تکرارپذیر نیستند.

راهبرد من این بود که برای کلاس‌هایی که تصویر بیشتر از هدف مقاله داشتند، با یک (SEED = 42) SubSampling انجام شد تا دقیقاً به عدد مقاله برسند (۱۱۶۱ کووید X-ray و ۴۲۴۰ ذات‌الریه) و برای کلاس‌هایی که تصویر کمتری داشتند، همه‌ی تصاویر موجود استفاده شد. در نتیجه، مجموعه داده‌ی نهایی بازتولید چنین شد:

mode	کلاس	هدف مقاله	Raw images	نهایی در شبیه سازی
X-ray	کووید-۱۹	۱۱۶۱	88%	۱۱۶۱ (SubSampling)
X-ray	ذات‌الریه	۴۲۴۰	86%	۴۲۴۰ (SubSampling)
X-ray	سالم	۳۸۰۰	۳۱۶۶	۳۱۶۶ (همه‌ی موجود)
X-ray	جمع	۹۲۰۱	—	8567
CT	کووید-۱۹	۱۳۰۵	۶۹۸	۶۹۸ (همه‌ی موجود)
CT	سالم	۱۴۵۷	۷۹۴	۷۹۴ (همه‌ی موجود)

mode	کلاس	هدف مقاله	Raw images	نهایی در شبیه سازی
CT	جمع	۲۷۶۲	—	1492

بنابراین در شبیه سازی من، مجموعه داده ی X-ray برابر 8567 تصویر (به جای ۹۲۰) و مجموعه داده ی CT برابر 1492 تصویر (به جای ۲۷۶۲) شد؛ یعنی به ترتیب حدود 93% و 54% حجم مقاله. کاهش حجم داده ی CT تقریباً به نصف، یکی از عوامل مهم در تفاوت نتایج نهایی است. نسبت کلاس ها در داده ی من همان الگوی نامتوازن مقاله را دارد.

=== Raw counts ===

```
xray_normal_dirs      : 3166
xray_pneumonia_dirs   : 8546
xray_covid_dirs       : 8088
ct_covid_dirs         : 698
ct_normal_dirs        : 794
```

Paper targets: X-ray COVID 1161 | Pneumonia 4240 | Normal 3800
CT COVID 1305 | Normal 1457

پیش پردازش – Preprocessing

این مراحل پیش پردازش ترکیب کار مقاله و پیاده سازی من است و هر جا که لازم بوده، تمایز این دو مشخص شده است.

نقشه ی کلی آماده سازی داده

مسیر آماده سازی داده در این پروژه از شش مرحله تشکیل شده است:

- **مرحله ۱:** تصاویر از سه منبع اصلی chest_xray، Mendeley، (UCSD COVID-CT) و شمارش تصاویر موجود در هر کلاس.

- **مرحله ی 2: یکسان سازی و نمونه گیری:** کپی تصاویر به یک ساختار پوشه ی یکپارچه (data_unified) ، Supsampling تا اعداد مقاله، خواندن همه ی تصاویر به صورت رنگی سه کاناله و تغییر اندازه ی همه به $224 \times 224 \times 3$.

```

• == Prepared counts (data_unified/) ==
• xray/COVID          : 1161
• xray/PNEUMONIA      : 4240
• xray/NORMAL         : 3166
• ct/COVID            : 698
• ct/NORMAL           : 794

```

- **مرحله ی 3: تقسیم بندی:** ساخت سه مجموعه ی آموزش/اعتبارسنجی/آزمون با تقسیم طبقه بندی شده (Stratified) و ذخیره ی آن ها در قالب فایل های CSV

- **مرحله ی 4: نرمال سازی:** نگاشت مقادیر پیکسل از بازه ی [0,255] به بازه ی [0,1] و تبدیل چینش کانال از HWC به CHW.

- **مرحله ی 5: افزایش داده:** اعمال چرخش تصادفی، جابه جایی کوچک و تغییر روشنایی مطابق جدول (مقاله) برای داده ی CT و در ادامه برای بهبود پایداری.

- **مرحله ی 6: تولید نویز مصنوعی:** افزودن سه نوع نویز گاوسی، نمک و فلفل و لکه ای با پنج سطح شدت به تصاویر تمیز برای ساخت جفت های (نویزی، تمیز) جهت آموزش اتوانکودر.

یکسان سازی، نمونه گیری و تغییر اندازه به $224 \times 224 \times 3$

این مرحله همه ی تصاویر X-ray و CT را به $224 \times 224 \times 3$ تغییر اندازه می دهد و ورودی معماری CNN فراهم میشود. این اندازه ی ورودی استاندارد خانواده ی معماری های کلاسیک VGG، ResNet، DenseNet و است که روی ImageNet آموزش دیده اند و چون بخشی از پروژه به یادگیری انتقالی از همین مدل ها است، انتخاب $224 \times 224 \times 3$ بیشترین سازگاری را با مدل های ازپیش آموزش دیده دارد.

در پیاده سازی من این مراحل انجام شد:

- هر تصویر ابتدا با OpenCV به صورت رنگی (IMREAD_COLOR) خوانده می شود و با کتابخانه ی PIL به RGB تبدیل و سپس استفاده می شود.
- تبدیل به سه کانال: چون معماری مقاله و مدل های ازپیش آموزش دیده ورودی سه کاناله می خواهند، همه ی تصاویر به فرمت رنگی سه کاناله RGB خوانده می شوند. در عمل کانال ها تکراری هستند اما ساختار تنسور با انتظار شبکه ها سازگار می ماند.
- تغییر اندازه با درون یابی: برای کوچک کردن تصاویر از درون یابی ناحیه ای (Area Interpolation) استفاده شد که برای کاهش ابعاد، کیفیت بهتری از درون یابی خطی یا نزدیک ترین همسایه دارد.
- ذخیره ی یکپارچه به فرمت PNG
- تابع آماده سازی طوری نوشته شد که اگر از قبل با تعداد کافی PNG معتبر وجود داشته باشد، از ساخت مجدد صرف نظر می کند.

تقسیم بندی train/val/test

- [۸۰:۲۰] به آموزش و آزمون تقسیم کرده و تصاویر را به صورت تصادفی انتخاب کرده است. شش فایل: سه نسبت برای X-ray و سه نسبت برای (CT)، اما با دو بهبود نسبت به آنچه در متن مقاله آمده:
- تقسیم طبقه بندی شده (Stratified Split): به جای تقسیم کاملاً تصادفی ساده، از تقسیم لایه ای بر اساس برچسب کلاس استفاده شد تا نسبت کلاس ها در هر سه مجموعه آموزش/اعتبارسنجی/آزمون دقیقاً حفظ شود. این تصمیم به دلیل نامتوازن بودن مجموعه ی X-ray حیاتی است.
 - افزودن مجموعه ی اعتبارسنجی (Validation Set): مقاله فقط از دو بخش آموزش و آزمون صحبت می کند، اما برای پیاده سازی درست توقف زودهنگام (Early Stopping) و انتخاب بهترین نقطه ی بازگشت وزن ها، 10% از داده ی آموزش به عنوان مجموعه ی اعتبارسنجی جدا شد.

بنابراین در نسبت ۸۰:۲۰ برای X-ray ۶۱۶۷ تصویر آموزش، ۶۸۶ اعتبارسنجی و ۱۷۱۴ آزمون و برای CT ۱۰۷۳ آموزش، ۱۲۰ اعتبارسنجی و ۲۹۹ آزمون.

Test	Val	Train	نسبت (آموزش:آزمون)	Mode
۱۷۱۴	۶۸۶	۶۱۶۷	۸۰:۲۰	X-ray
۲۹۹	۱۲۰	۱۰۷۳	۸۰:۲۰	CT

Normalization

از ReLU به عنوان تابع فعال سازی در همه ی لایه ها و Softmax در لایه ی خروجی استفاده شده است.

Data Augmentation

چون مجموعه های CT کوچک اند و ممکن است باعث overfit شوند، یک فرآیند افزایش داده روی مجموعه ی CT انجام شده است. این افزایش داده شامل تغییراتی مانند

- تغییر عرض تصویر و چرخش تصویر است
- بازه ی روشنایی و اعمال افزایش برای هر بخش از برش روی تصاویر دیجیتال به طور مناسب برای هر نمونه ی آموزشی بررسی شده است.

جدول مقاله Augmentation داده را چنین مشخص می کند:

پارامتر	مقدار در مقاله	توضیح
Width shift range	2	ضریب جابه‌جایی افقی تصویر
Rotation range	4	بازه‌ی درجه‌ی چرخش تصادفی
Brightness	(0.5 و 1.5)	بازه‌ی انتخاب ضریب تغییر روشنایی
Sample-wise center	True	صفرکردن میانگین هر نمونه
Feature-wise center	True	صفرکردن میانگین ورودی در کل مجموعه داده، به صورت ویژگی‌محور

RandomAffine: با درجه‌ی چرخش ۴ و جابه‌جایی افقی معادل ۲ پیکسل (تطبیق‌یافته با مقیاس ۲۲۴) و ColorJitter با بازه‌ی روشنایی (0.5 و 1.5).

تولید نویز مصنوعی و ساخت جفت‌های نویزی/تمیز

اتوانکودر نویز‌زدا به جفت تصویر نیاز دارد: یک تصویر نویزی به عنوان ورودی و نسخه‌ی تمیز همان تصویر به عنوان هدف. (Target) مقاله سه نوع نویز را با پنج سطح شدت بررسی می‌کند و من دقیقاً همان ماتریس ۵۵ تایی (۳ نوع 5×5 سطح) را برای هر دو نوع تصویر پیاده کردم (مجموعاً ۳۰ پیکربندی):

- **نویز گاوسی (Gaussian):** نویز جمع‌شونده‌ی با میانگین صفر و انحراف معیار $\sqrt{var}$ که به صورت مستقل به هر پیکسل و هر کانال اضافه می‌شود و سپس تصویر به بازه‌ی $[0,1]$ بریده (Clip) می‌شود. این نویز مدل استاندارد نویز الکترونیکی حسگرهای تصویربرداری است.
- **نویز نمک و فلفل (Salt & Pepper):** با چگالی d ، تعداد متناظری پیکسل به صورت تصادفی انتخاب می‌شوند؛ نیمی به سفید مطلق (1.0) و نیمی به سیاه مطلق (0.0) تبدیل می‌شوند. این نویز مدل خطاهای تلفات بیتی و نقاط خراب حسگر است و برخلاف نویز گاوسی، ماهیتی گسسته و دامنه‌دار دارد.

- **نویز لکه‌ای (Speckle):** نویز ضرب‌شونده‌ای است که در آن شدت نویز به خود مقدار پیکسل بستگی دارد. این نویز در تصویربرداری پزشکی به‌ویژه سونوگرافی و CT بسیار متداول است و چون با سیگنال همراه است، حذف آن دشوارتر از نویز جمع‌شونده است.

پنج سطح شدت مورد استفاده، دقیقاً مطابق مقاله 0.05، 0.10، 0.15، 0.20 و 0.25 هستند. یک نکته‌ی دقیق پیاده‌سازی: نویز به‌صورت در لحظه و در هر بار خواندن نمونه از Dataset تولید می‌شود، نه اینکه تصاویر نویزی از پیش روی دیسک ذخیره شوند. این کار دو مزیت دارد: اول، فضای ذخیره‌سازی چندبرابر نمی‌شود؛ و دوم، در هر Epoch نمونه‌ی نویزی متفاوتی از همان تصویر تولید می‌شود که عملاً نقش یک افزایش داده‌ی پنهان را هم ایفا می‌کند و اتوانکودر را به‌جای حفظ کردن یک نگاشت ثابت، به یادگیری مفهوم نویز وادار می‌کند.

مراحل پیاده‌سازی در مقاله

مقاله ساخت مدل نهایی را در هشت گام خلاصه می‌کند:

- **گام‌های ۱ و ۲ (آموزش CNN ها):** مدل‌های یادگیری انتقالی و FCNN ساخته و با تابع هزینه‌ی آنتروپی آموزش داده می‌شوند و وزن‌هایشان ذخیره می‌گردد. نکته: این آموزش روی تصاویر اصلی انجام می‌شود و نتایجش همان نتایج بدون AD
- **گام‌های ۳ و ۴ (آموزش اتوانکودر):** شبکه‌ی اتوانکودر نویززا ساخته و با تابع هزینه‌ی MSE آموزش داده می‌شود و وزن‌هایش ذخیره می‌گردد. ورودی این آموزش، جفت‌های (نویزی، تمیز) است.

- **گام ۵ Classification مجدد پس از AD:** مدل‌های یادگیری انتقالی و FCNN این بار روی تصاویری آموزش/ارزیابی می‌شوند که از اتوانکودر عبور کرده‌اند.
- **گام ۶ CADTra:** اتوانکودر و طبقه‌بند در قالب یک شبکه‌ی ترکیبی واحد سازماندهی مجدد می‌شوند. یعنی خروجی اتوانکودر مستقیماً ورودی طبقه‌بند می‌شود.
- **گام ۷ Freeze وزن‌های AD:** وزن‌های اتوانکودر ثابت نگه داشته می‌شوند؛ یعنی اتوانکودر در این مرحله دیگر آموزش نمی‌بیند. این تصمیم از تخریب وزن‌های AE توسط گرادیان‌های Classification جلوگیری می‌کند.
- **گام ۸ (Fine-Tuning):** تمام پارامترهای شبکه‌ی کلی بر اساس وزن‌های اتوانکودر و مدل انتقالی Fine tune می‌شوند و مدل نهایی CADTra ذخیره می‌گردد.

مقاله Hyperparameters

Category	Hyperparameter	Value
Global	Random seed (SEED)	42
Global	Image size (IMG_SIZE)	224 x 224 x 3
Global	cudnn.benchmark	True
Global	Device	auto (cuda/cpu)

Category	Hyperparameter	Value
Data	Noise types	[gaussian, salt_pepper, speckle]
Data	Noise variances	[0.05, 0.1, 0.15, 0.2, 0.25]
Data	Train/test splits	[80/20, 70/30, 60/40]
Data	X-ray targets (COVID/PNEUMONIA/NORMAL)	1161 / 4240 / 3800
Data	CT targets (COVID/NORMAL)	1305 / 1457
Denoising AE	Learning rate (AD_LR)	0.0002
Denoising AE	Adam betas (AD_B1, AD_B2)	(0.5, 0.999)
Denoising AE	Adam epsilon (AD_EPS)	0.01
Denoising AE	Epochs (AE_TRAIN_EPOCHS, paper = 20)	20
Denoising AE	Batch size (AD_BATCH / AD_BATCH_FAST)	16
Denoising AE	Early-stop patience (AE_PATIENCE)	3
Denoising AE	Train subset fraction (SUBSET_FRAC)	1.0

Category	Hyperparameter	Value
Denoising AE	Eval samples per config (EVAL_SAMPLES)	75
Denoising AE	Loss	MSE (modified Gaussian likelihood)
Denoising AE	Optimizer	Adam
Denoising AE	Encoder filters	3 -> BN -> 128 -> 64 -> 32 (latent)
Denoising AE	Decoder filters	32 -> 64 -> 128 -> 3 (sigmoid)
Denoising AE	Kernel / stride / padding	3x3 / stride 1 / same (=1)
Denoising AE	Mixed precision (AMP)	enabled on CUDA
Classifier (paper)	Learning rate (CLS_LR)	0.00001
Classifier (paper)	Adam betas (CLS_B1, CLS_B2)	(0.9, 0.999)
Classifier (paper)	Epochs (CLS_EPOCHS / CLS_EPOCHS_FAST)	30
Classifier (paper)	Batch size (CLS_BATCH)	16

Category	Hyperparameter	Value
Classifier (paper)	Early-stop patience (CLS_PATIENCE)	4
Classifier (paper)	Head learning rate	CLS_LR * 10
Classifier (paper)	Loss	CrossEntropyLoss
Classifier (paper)	Optimizer	Adam
Classifier (paper)	LR schedule (head-on-features)	StepLR(step=2, gamma=0.5)
Classifier (CT fix)	CT FCNN LR (CT_FCNN_LR)	0.0003
Classifier (CT fix)	CT FCNN epochs (CT_FCNN_EPOCHS)	40
Classifier (CT fix)	CT FCNN patience (CT_FCNN_PATIENCE)	8
Classifier (CT fix)	CT FCNN dropout (CT_FCNN_DROPOUT)	(0.10, 0.05)
Classifier (CT fix)	CT FCNN weight decay	0.0001
Classifier (CT fix)	CT FCNN LR schedule	CosineAnnealingLR(T_max=40)

Category	Hyperparameter	Value
Classifier (CT fix)	CT head epochs (CT_HEAD_EPOCHS_AUG)	20
Classifier (CT fix)	CT head patience (CT_HEAD_PATIENCE)	6
Classifier (CT fix)	CT head LR schedule	StepLR(step=4, gamma=0.5)
Transfer learning	Use ImageNet pretrained (USE_PRETRAINED)	True
Transfer learning	Freeze backbone (FREEZE_BACKBONE)	True
Transfer learning	Pretrained weights	IMAGENET1K_V1
Transfer learning	Batches (FCNN/VGG16/ResNet50/DenseNet121)	32 / 32 / 16 / 16
Transfer learning	Head batch (feature-cache path)	256
Transfer learning	Feature-extract batch (VGG16/ResNet50/Dense)	32 / 32 / 32
Transfer learning	CT augmentation	RandomAffine(4 deg) + ColorJitter(0.5-1.5)

معیارهای ارزیابی مقاله

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (4)$$

$$Precision = \frac{TP}{TP + FP} \quad (5)$$

$$Recall = \frac{TP}{TP + FN} \quad (6)$$

$$f_1\text{-score} = 2 \times \frac{Recall + Precision}{Recall \times Precision} \quad (7)$$

$$\log loss = -\log P(y_T/y_P) = -y_T \log(y_P) + (1 - y_T) \log(1 - y_P) \quad (8)$$

$$PSNR = 10 \log_{10} \left(\frac{MAX_I^2}{MSE} \right) \quad (9)$$

$$SSIM(x, y) = \frac{(2\mu_x\mu_y + c_1)(2\sigma_{xy} + c_2)}{(\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2)} \quad (10)$$

شبیه سازی گام به گام:

خلاصه کلی از مراحل پروژه و روند بازسازی من در فایل کد:

گام	محتوا	گام متناظر در مقاله
STEP 0	ایمپورت‌ها، تنظیم Cupy، seed، تعریف ابرپارامترها و مسیرها	بخش ۳-۴ مقاله
STEP 1	مکان‌یابی پوشه‌های داده و شمارش تصاویر	شروع بخش ۱-۴
STEP 2	Subsampling و تغییر اندازه به $۲۲۴ \times ۲۲۴ \times ۳$	بخش ۳ (پیش‌پردازش)
STEP 3	ساخت سه نسبت تقسیم آموزش/اعتبارسنجی/آزمون به CSV	بخش ۱-۴
STEP 4	توابع نویز و PSNR/SSIM؛ نمایش نمونه	معیارهای بخش ۴-۴
STEP 5	کلاس‌های Dataset و DataLoader برای طبقه‌بند و اتوانکودر	زیرساخت داده
STEP 6	پیاده‌سازی دقیق اتوانکودر مقاله در PyTorch از صفر	بخش ۱-۳ و شکل ۲
STEP 6b	خلاصه‌ی معماری همه‌ی ۱۳ شبکه با torchinfo	مستندسازی
STEP 7	آموزش ۳۰ اتوانکودر (۲ مدالیت ۳×۳ نویز ۵×۵ واریانس)	گام‌های ۳-۴ مقاله
STEP 8	ساخت مقایسه‌ی PSNR/SSIM	جدول ۵ مقاله
STEP 9	نمودارهای PSNR/SSIM: بار، خط، حرارتی	ارزیابی AD

گام	محتوا	گام متناظر در مقاله
STEP 10	گالری تمیز/نویزی/نویززدایی شده	کیفی سازی AD
STEP 11	پیاده سازی FCNN	بخش ۲-۳
STEP 12	آموزش طبقه بندها روی داده ی نویززدایی شده (فاز CADTra)	گام های ۵-۸ مقاله
STEP 13	نمودارهای CM/ROC/PR برای هر مدل	شکل های جدول ۴ مقاله
STEP 13b	آموزش طبقه بندها روی داده ی اصلی (بدون AD، فاز مبنا)	گام های ۱-۲ مقاله
STEP 14	خلاصه ی نهایی نتایج و نمودارها	جمع بندی
STEP 15	خروجی های سبک مقاله (جدول ها، منحنی های آموزش، داشبورد)	شکل های نهایی

سه فاز آزمایش: قبل از نویززدایی، نویززدایی، بعد از نویززدایی

(۱) طبقه بندی روی داده ی اصلی، (۲) آموزش و ارزیابی اتوانکودر و (۳) طبقه بندی روی داده ی نویززدایی شده. سپس نتایج فاز اول و سوم مقایسه می شوند تا اثر واقعی نویززدایی روی طبقه بندی سنجیده شود.

فاز اول: Classification روی تصاویر اصلی (بدون AD)

در این فاز که در سلول STEP 13b نوت‌بوک پیاده شد، هر سه مدل (ResNet50 و VGG16، FCNN) روی تصاویر اصلی (بدون هیچ‌گونه پردازش نوینزدایی) با نسبت ۸۰:۲۰ آموزش دیدند و روی مجموعه‌ی آزمون ارزیابی شدند. این فاز به سوال «مدل بدون AD چقدر خوب کار می‌کند؟» پاسخ می‌دهد.

Modality	مدل	Accuracy	F1	Log-Loss
X-ray	FCNN	%87.98	%83.56	۰/۳۷۷۴
X-ray	VGG16 (پیش‌آموزش، Frozen Body)	%93.12	%89.25	۰/۲۲۴۷
X-ray	ResNet50 (پیش‌آموزش، Frozen Body)	%89.67	%86.37	۰/۳۲۹۵
CT	FCNN	%89.63	%89.53	۰/۳۶۰۸
CT	VGG16 (پیش‌آموزش، Frozen Body)	%91.97	%91.89	۰/۳۳۱۱
CT	ResNet50 (پیش‌آموزش، Frozen Body)	%72.58	%72.46	۰/۵۴۱۳

بهترین مدل روی X-ray: VGG16 با دقت %93.12 (در مقاله بدون AD: %95.74 برای VGG16 پیش‌آموزش)

بهترین مدل روی CT: VGG16 با دقت %91.97 (در مقاله بدون AD: %94.76)

FCNN روی X-ray و CT به ترتیب %87.98 و %89.63 گرفت که به طرز محسوسی از عدد مقاله (%98.38 و %97.64) پایین‌تر است. این یکی از بزرگ‌ترین تفاوت‌ها است که پیش‌بینی می‌کنم برمیگردد به عدم دسترسی به دقیقاً همان مجموعه داده‌ی مقاله و تصادفی بودن تقسیم مربوط است.

فاز دوم: آموزش و ارزیابی اتوانکودرهای نویززا

در این فاز که در سلول STEP 7 پیاده شد، ۳۰ اتوانکودر مستقل آموزش داده شدند: برای هر یک از ۲ مدالیت (CT و X-ray)، هر یک از ۳ نوع نویز (گوسی، نمک و فلفل، لکه ای) و هر یک از ۵ سطح واریانس، یک اتوانکودر مستقل با ابرپارامترهای مقاله آموزش می یابند. نتایج:

Noise	σ	Paper PSNR	My PSNR	% of paper	Paper SSIM	My SSIM	% of paper
Gaussian	0.05	37.30	30.01	80.5%	0.9469	0.8093	85.5%
Gaussian	0.10	34.29	28.69	83.7%	0.9063	0.7646	84.4%
Gaussian	0.15	31.66	27.71	87.5%	0.8715	0.7471	85.7%
Gaussian	0.20	31.06	27.13	87.3%	0.8444	0.7252	85.9%
Gaussian	0.25	29.86	26.64	89.2%	0.8309	0.7084	85.3%
Salt&Pepper	0.05	41.77	35.69	85.4%	0.9801	0.9406	96.0%
Salt&Pepper	0.10	39.01	35.19	90.2%	0.9824	0.9358	95.3%

Noise	σ	Paper PSNR	My PSNR	% of paper	Paper SSIM	My SSIM	% of paper
Salt&Pepper	0.15	39.60	33.82	85.4%	0.9758	0.9223	94.5%
Salt&Pepper	0.20	35.08	33.98	96.9%	0.9792	0.9279	94.8%
Salt&Pepper	0.25	38.11	31.79	83.4%	0.9779	0.8899	91.0%
Speckle	0.05	32.11	32.48	101.2%	0.8979	0.8811	98.1%
Speckle	0.10	32.11	30.48	94.9%	0.8740	0.8327	95.3%
Speckle	0.15	31.73	29.39	92.6%	0.8669	0.7954	91.8%
Speckle	0.20	31.10	30.68	98.6%	0.8543	0.8390	98.2%
Speckle	0.25	30.38	29.84	98.2%	0.8433	0.8218	97.5%

gaussian
clean



var=0.05
PSNR=13.9



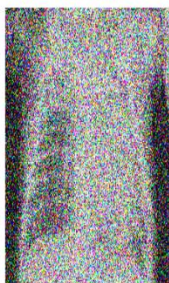
var=0.1
PSNR=11.6



var=0.15
PSNR=10.4



var=0.2
PSNR=9.6



var=0.25
PSNR=9.2



salt_pepper
clean



var=0.05
PSNR=15.3



var=0.1
PSNR=12.5



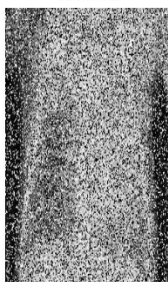
var=0.15
PSNR=10.9



var=0.2
PSNR=9.8



var=0.25
PSNR=9.0



speckle
clean



var=0.05
PSNR=17.5



var=0.1
PSNR=15.0



var=0.15
PSNR=13.6



var=0.2
PSNR=12.6



var=0.25
PSNR=11.9



► CADTra Autoencoder (input 224x224x3)

```
=====
=====
Layer (type:depth-idx)           Output Shape           Param #
=====
=====
CADTraAutoencoder               [1, 3, 224, 224]       --
├─BatchNorm2d: 1-1               [1, 3, 224, 224]       6
├─Conv2d: 1-2                    [1, 128, 224, 224]     3,584
├─Conv2d: 1-3                    [1, 64, 224, 224]      73,792
├─Conv2d: 1-4                    [1, 32, 224, 224]      18,464
├─ConvTranspose2d: 1-5           [1, 32, 224, 224]      9,248
├─ConvTranspose2d: 1-6           [1, 64, 224, 224]      18,496
├─ConvTranspose2d: 1-7           [1, 128, 224, 224]     73,856
├─Conv2d: 1-8                   [1, 3, 224, 224]       3,459
=====
=====
Total params: 200,905
Trainable params: 200,905
Non-trainable params: 0
Total mult-adds (Units.GIGABYTES): 10.08
=====
=====
Input size (MB): 0.60
Forward/backward pass size (MB): 182.24
Params size (MB): 0.80
Estimated Total Size (MB): 183.65
=====
=====
```

فاز سوم: طبقه‌بندی روی تصاویر نويززدایی‌شده (CADTra)

در این فاز که در سلول STEP 12 نوت‌بوک پیاده شد، ابتدا تمام تصاویر مجموعه داده از اتوانکودر عبور داده می‌شوند و خروجی نويززدایی‌شده روی دیسک کش می‌گردد. سپس دقیقاً همان سه مدل (VGG16، FCNN و ResNet50) با همان تنظیمات فاز اول، اما این بار روی داده‌ی نويززدایی‌شده آموزش داده و ارزیابی می‌شوند. نتایج:

Modality	مدل	Accuracy	Precision	Recall	F1
X-ray	FCNN	%87.28	%88.57	%80.66	%83.31
X-ray	VGG16	%92.30	%93.80	%86.04	%88.81
X-ray	ResNet50	%89.03	%91.15	%82.74	%85.57
CT	FCNN	%89.97	%90.34	%89.67	%89.86
CT	VGG16	%91.30	%91.98	%90.93	%91.18
CT	ResNet50	%55.52	%60.90	%57.37	%52.45

جمع بندی عددی قبل و بعد از نویززدایی

Modality	مدل	Acc بدون AD	Acc با AD	Δ Acc	F1 بدون AD	F1 با AD	Δ F1
X-ray	FCNN	%87.28	%87.98	-%0.70	%83.56	%83.31	-%0.25
X-ray	VGG16	%92.30	%93.12	-%0.82	%89.25	%88.81	-%0.44
X-ray	ResNet50	%89.03	%89.67	-%0.64	%86.37	%85.57	-%0.80
CT	FCNN	%89.97	%89.63	+%0.34	%89.53	%89.86	+%0.33
CT	VGG16	%91.30	%91.97	-%0.67	%91.89	%91.18	-%0.71
CT	ResNet50	%55.52	%72.58	-%17.06	%72.46	%52.45	-%20.01

تفاوت‌ها با مقاله، توجیه تصمیم‌ها و تحلیل اثر آن‌ها بر نتایج

مؤلفه	در مقاله	در شبیه سازی من	وضعیت
معماری اتوانکودر (۸ لایه)	BN + Conv(128,64,32) + ConvTrans(32,64,128) + Conv (3)	عیناً همان (۲۰۰,۹۰۵) پارامتر	مطابق
معماری (۴FCNN + ۳Conv + FC)	Pool + ۲Conv (16,32,64,64) + ۴BN + (0.15/0.22)FC + Dropout ۳	عیناً همان (۲۵,۷۵۹,۲۰۹) پارامتر	مطابق
ابعاد ورودی	۳×۲۲۴×۲۲۴	عیناً همان	مطابق
سه نوع نویز × ۵ واریانس	گوسی، نمک و فلفل، لکه‌ای × {۲۵/۰...۰۵/۰}	عیناً همان	مطابق
تابع هزینه‌ی اتوانکودر	$\log N(y; \hat{y}, \sigma^2) \propto \text{MSE}-$	MSE (معادل ریاضی همان)	مطابق
بهینه‌ساز اتوانکودر	(0.5,0.999)=Adam, lr=2e-4, β	عیناً همان	مطابق
بهینه‌ساز طبقه‌بند	(0.9,0.999)=Adam, lr=1e-5, β	عیناً همان + StepLR (گام $\gamma=0.5, ۲$)	مطابق
توقف زودهنگام	صبر ۴ Epoch	صبر ۴ (X-ray FCNN)، صبر ۳ (AE)، صبر ۶-۸ (CT)	تقریبی
منابع داده	UCSD COVID-CT + Mendeley + Kaggle chest_xray	عیناً همان سه منبع	مطابق

مؤلفه	در مقاله	در شبیه سازی من	وضعیت
حجم داده	۲۷۶۲CT: ،۹۲۰IX-ray:	۱۴۹۲CT: ،۸۵۶۷X-ray:	متفاوت
تعداد مدل‌ها	۱۳ معماری	۳ معماری (FCNN) ، (ResNet50 ،VGG16)	متفاوت
سخت افزار	GB RAM۱۳NVIDIA TESLA P100 +	GB۴RTX 3050 Laptop	متفاوت
تعداد Epoch اتوانکودر	گزارش نشده (با توقف زودهنگام)	حداکثر ۲۰ (با صبر ۳)	تقریبی
تعداد Epoch طبقه‌بند	گزارش نشده	حداکثر ۳۰ (با صبر ۴)	تقریبی
افزایش داده	فقط روی CT	CT + سرهای پیش آموزش (در هر دو مُدالیت)	متفاوت
نویز مورد استفاده در CADTra	گزارش نشده صریحاً	گاوسی = ۱۰/۰۷ (یک ترکیب)	متفاوت

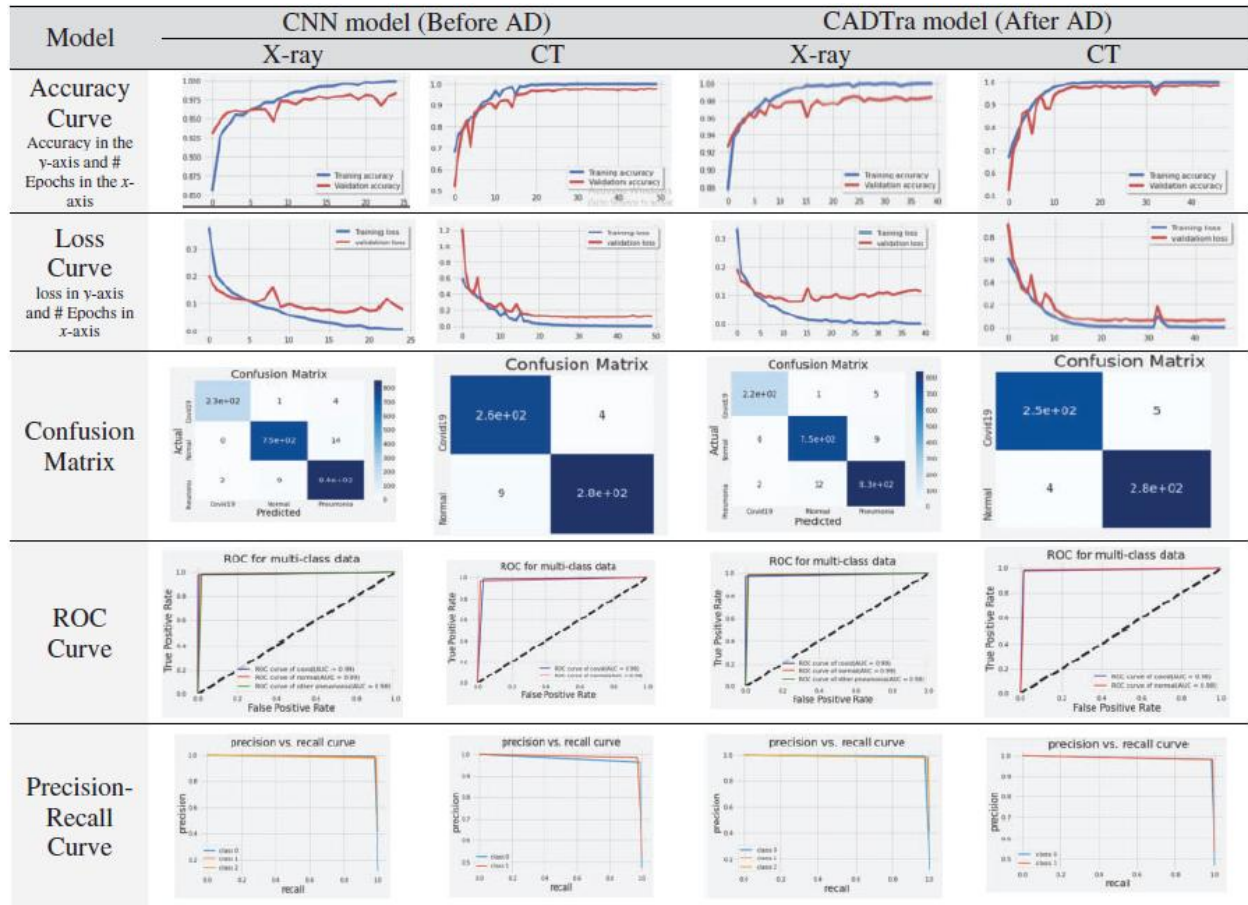
مقایسه نهایی درصد ها:

accuracy		f1 \		
variant		denoised_with_AD	original_no_AD	denoised_with_AD
modality	model			
ct	FCNN	0.8997	0.8963	0.8986
	ResNet50	0.5552	0.7258	0.5245
	VGG16	0.9130	0.9197	0.9118
xray	FCNN	0.8728	0.8798	0.8331
	ResNet50	0.8903	0.8967	0.8557
	VGG16	0.9230	0.9312	0.8881

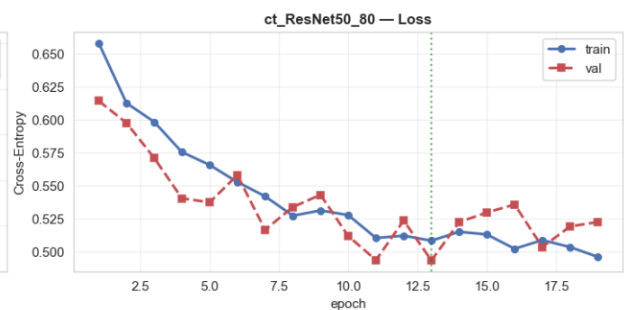
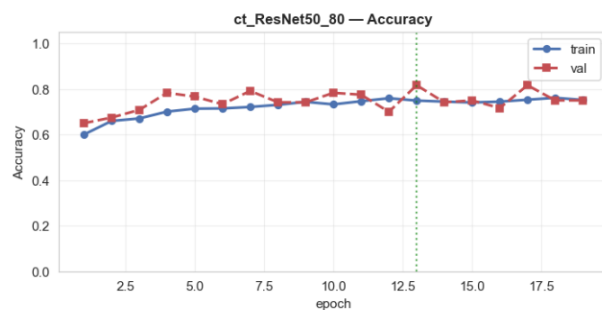
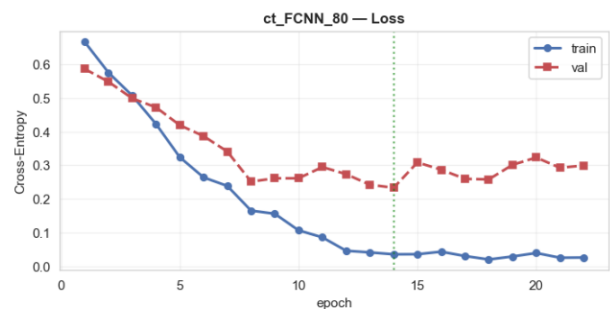
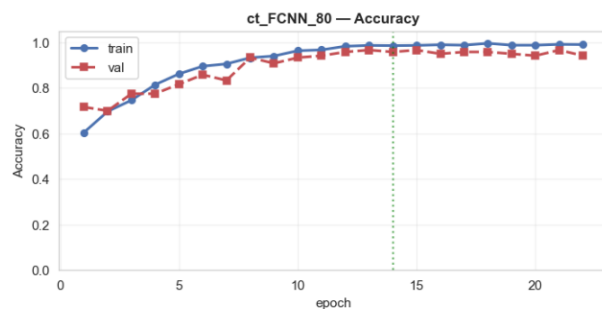
		log_loss		
variant		original_no_AD	denoised_with_AD	original_no_AD
modality	model			
ct	FCNN	0.8953	0.5558	0.3608
	ResNet50	0.7246	0.7217	0.5413
	VGG16	0.9189	0.4314	0.3311
xray	FCNN	0.8356	0.3986	0.3774
	ResNet50	0.8637	0.3367	0.3295
	VGG16	0.8925	0.2541	0.2247

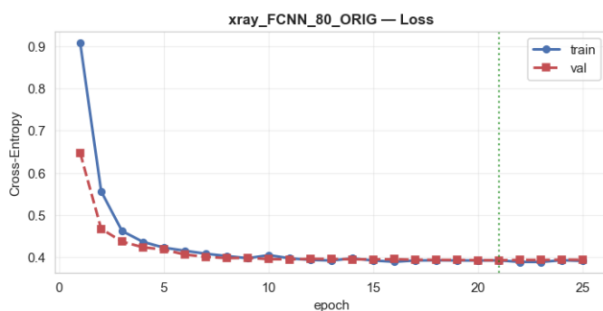
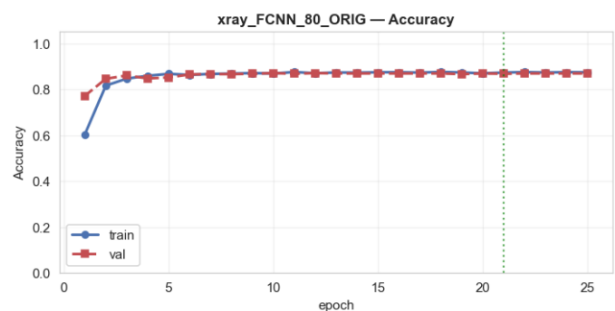
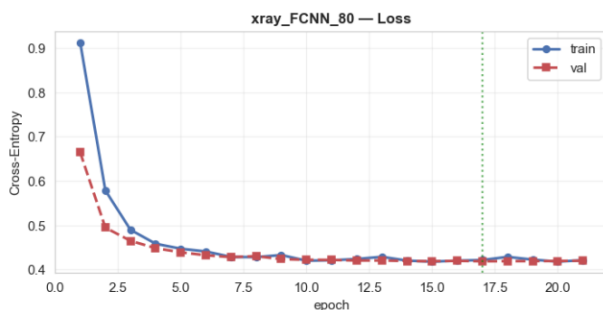
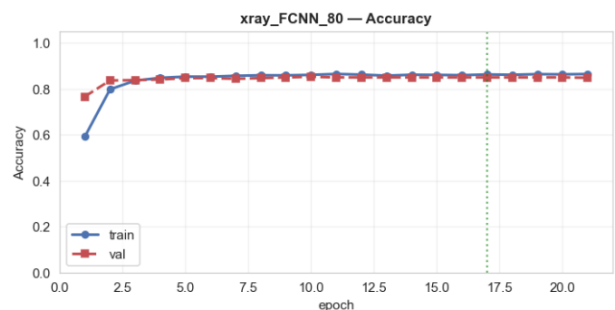
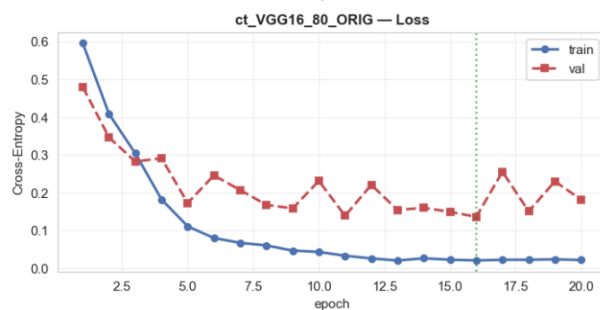
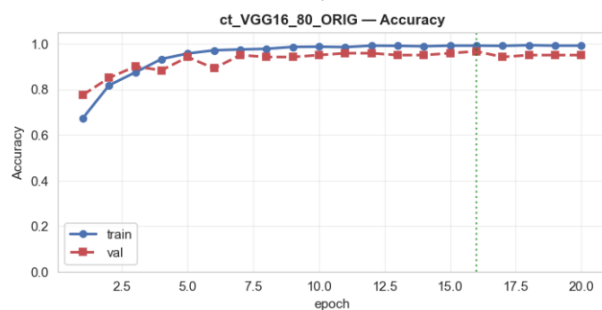
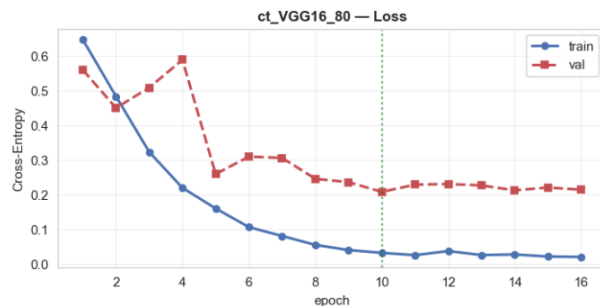
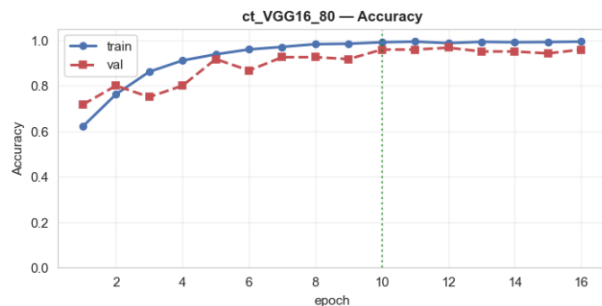
خروجی ها:

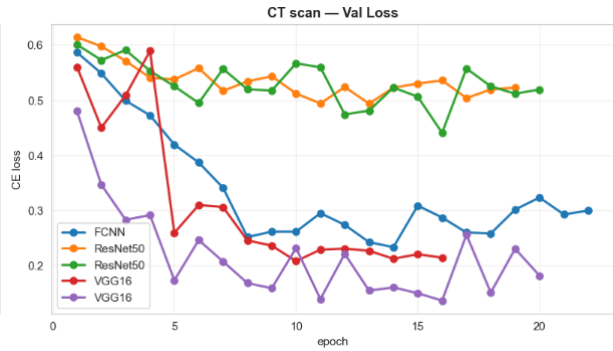
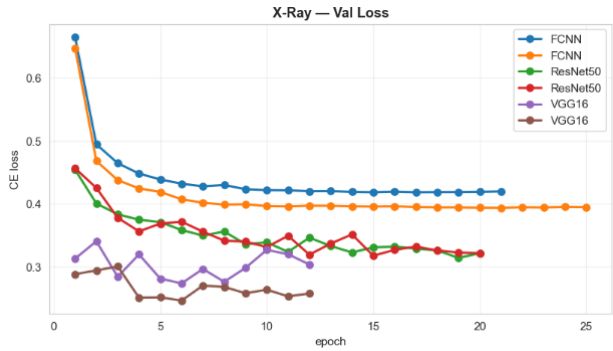
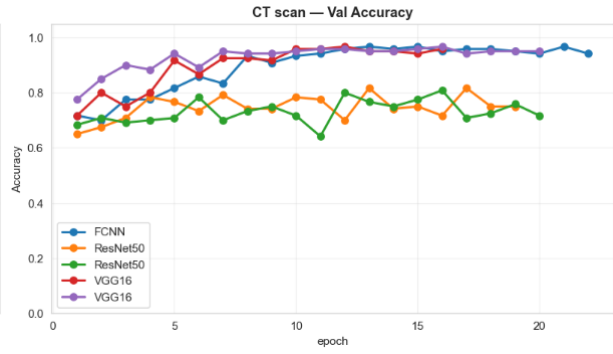
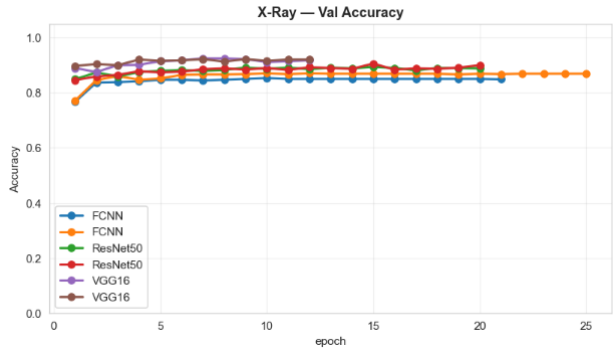
خروجی های خود مقاله



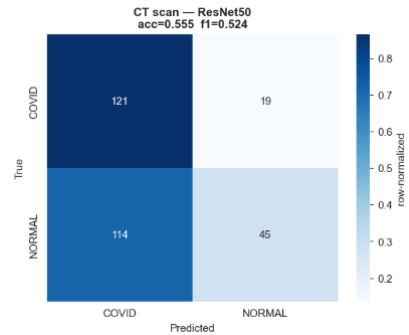
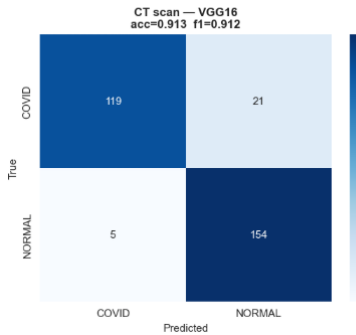
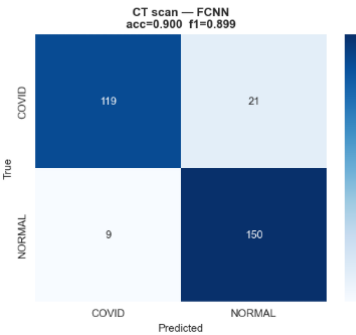
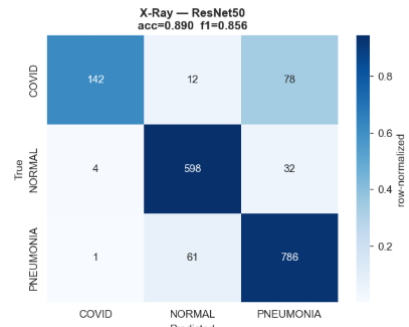
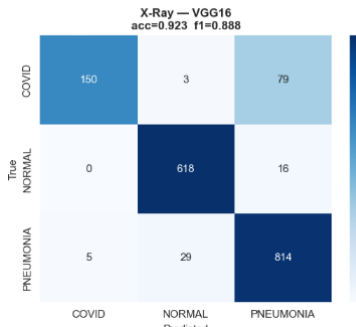
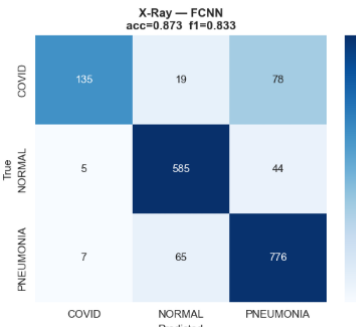
Noise	Variance	CT scan PSNR	X-Ray PSNR	CT scan SSIM	X-Ray SSIM
Gaussian	0.05	23.4735	30.0112	0.7051	0.8093
Gaussian	0.1	24.1528	28.6865	0.6648	0.7646
Gaussian	0.15	23.1371	27.7091	0.6363	0.7471
Gaussian	0.2	23.3702	27.1324	0.6221	0.7252
Gaussian	0.25	22.2051	26.6354	0.5882	0.7084
Salt & Pepper	0.05	21.904	35.6941	0.6635	0.9406
Salt & Pepper	0.1	23.4766	35.1922	0.6685	0.9358
Salt & Pepper	0.15	22.4251	33.8215	0.6622	0.9223
Salt & Pepper	0.2	20.4764	33.9829	0.5526	0.9279
Salt & Pepper	0.25	20.0815	31.7909	0.5276	0.8899
Speckle	0.05	25.5848	32.4812	0.775	0.8811
Speckle	0.1	26.339	30.4819	0.7541	0.8327
Speckle	0.15	25.1387	29.3924	0.7303	0.7954
Speckle	0.2	24.8997	30.6804	0.7237	0.839
Speckle	0.25	23.8829	29.8442	0.6916	0.8218

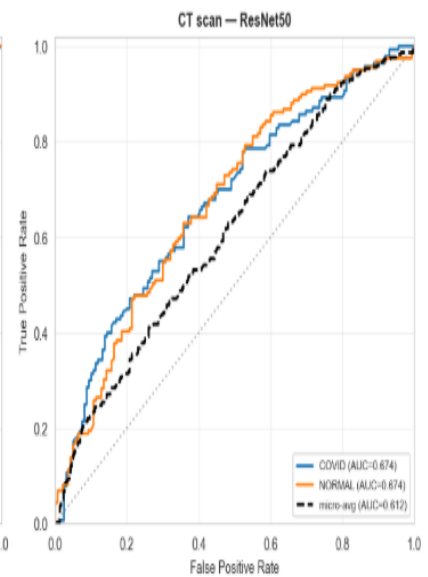
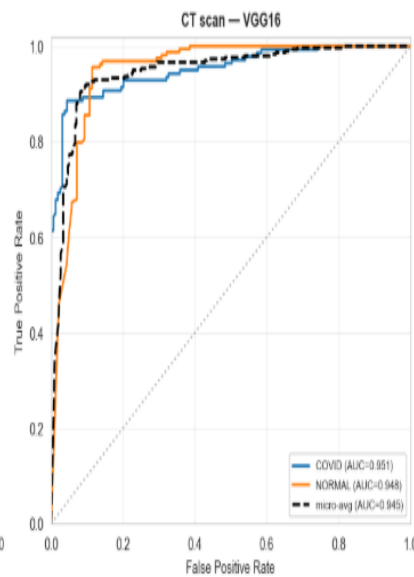
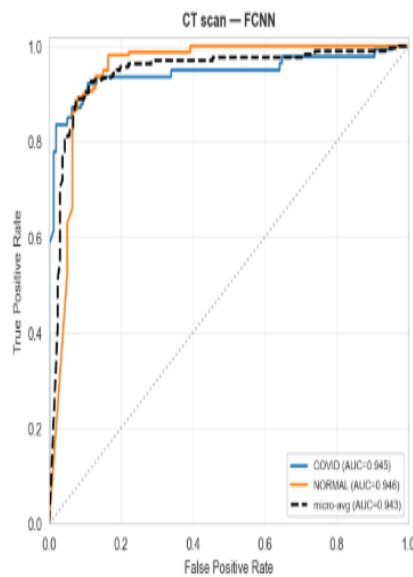
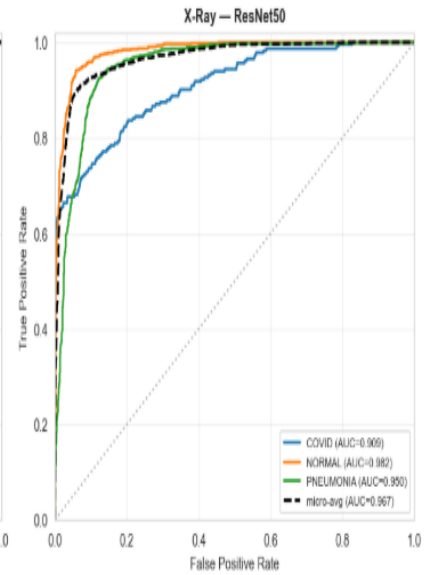
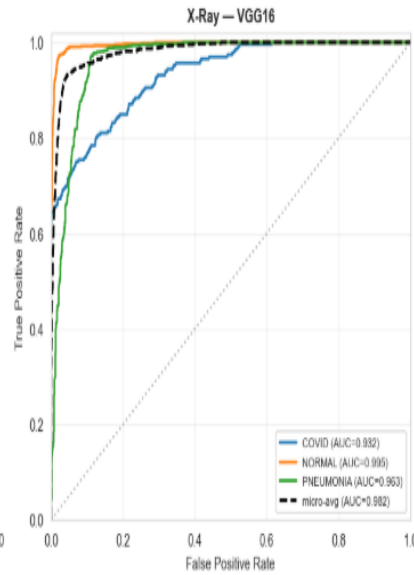
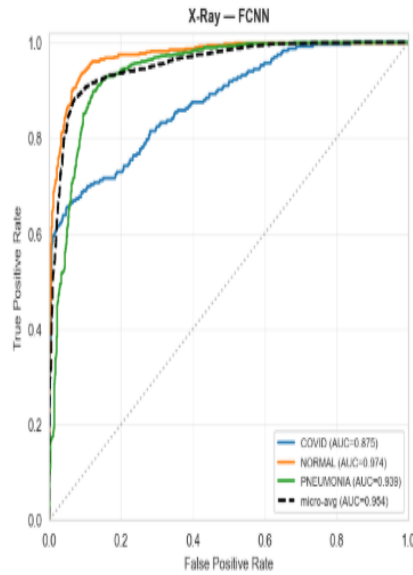


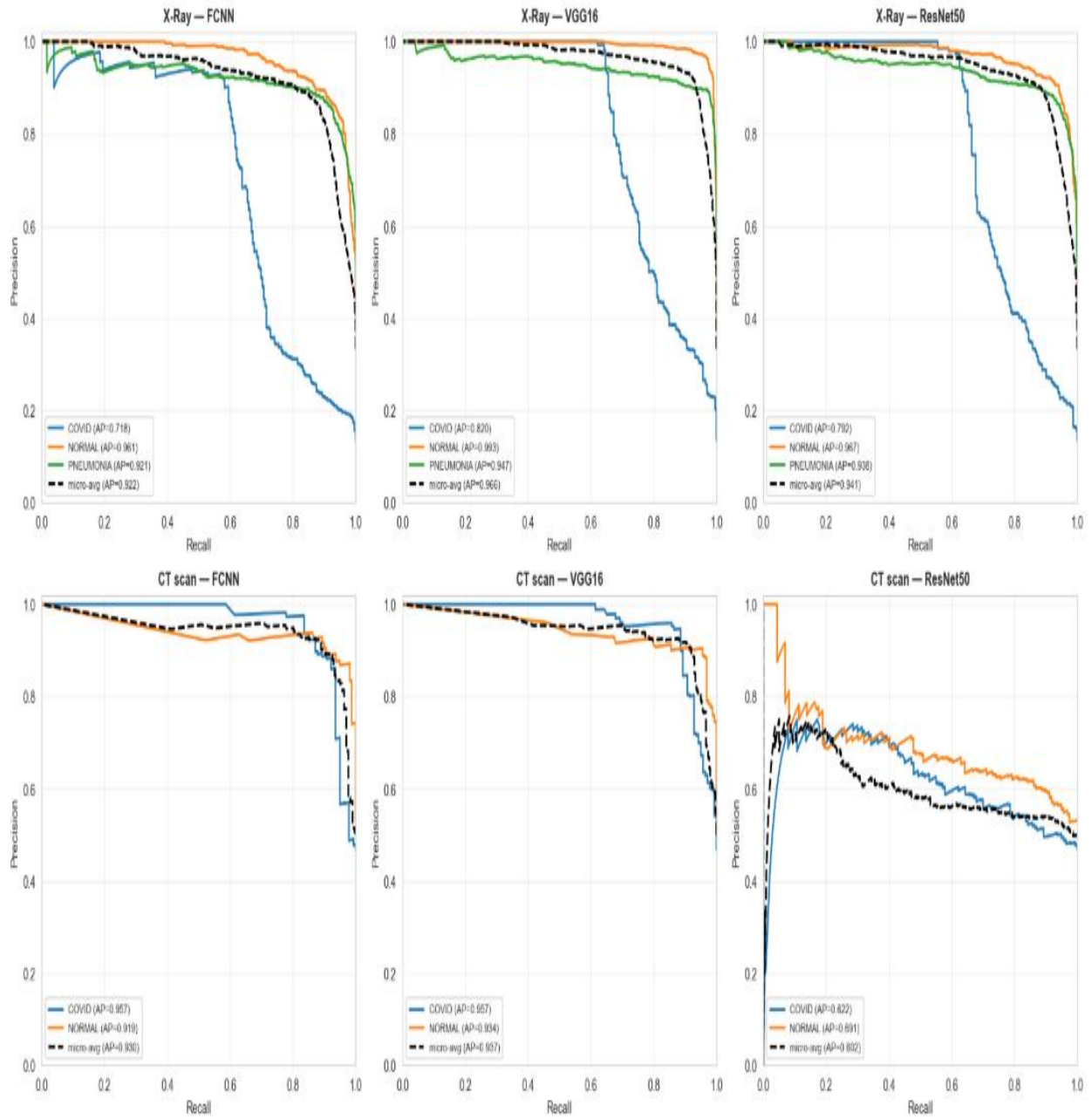


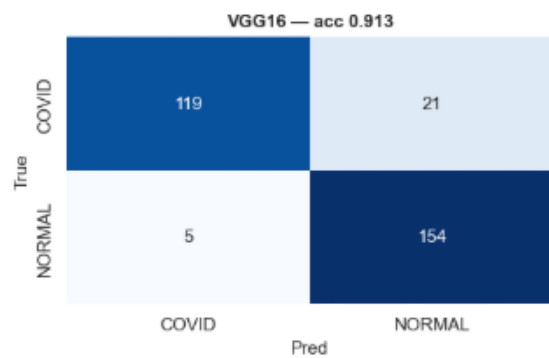
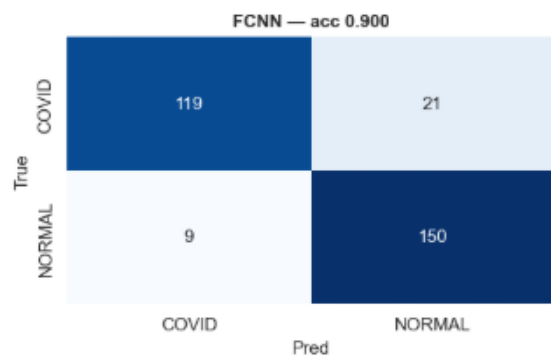
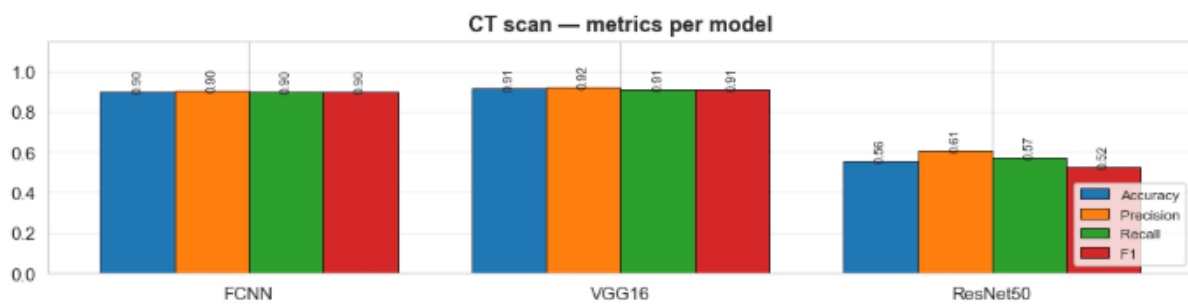


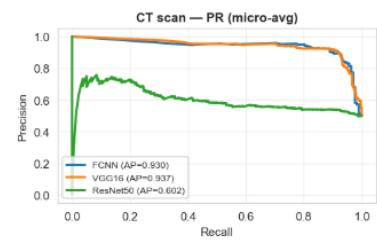
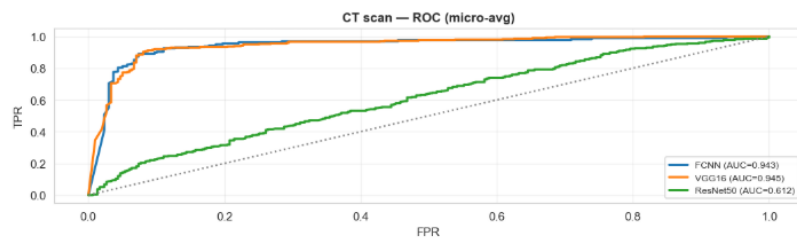
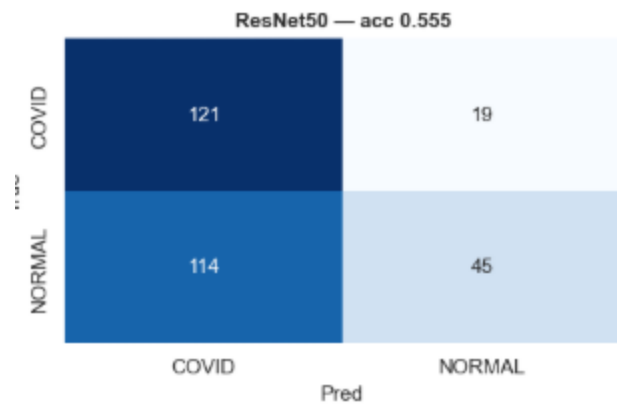
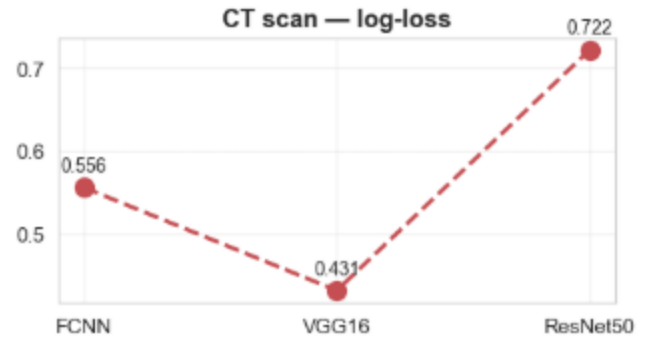
Our Confusion Matrices — counts annotated, color = row %



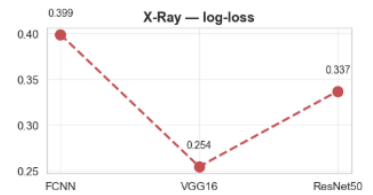
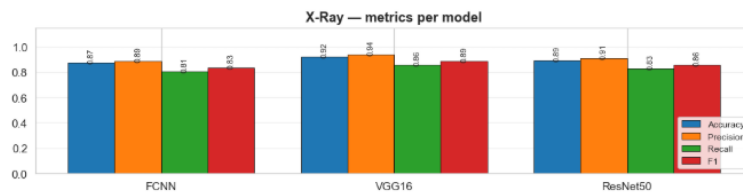








Our Full Dashboard — X-Ray

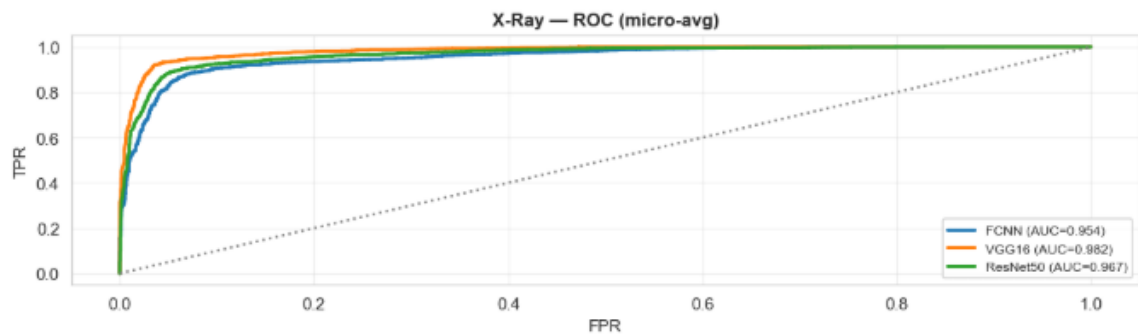


FCNN — acc 0.873

	COVID	NORMAL	PNEUMONIA
True COVID	135	19	78
True NORMAL	5	585	44
True PNEUMONIA	7	65	776
	COVID	NORMAL	PNEUMONIA
	Pred		

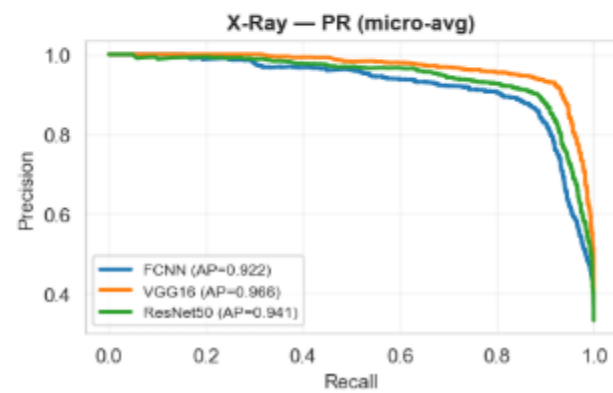
VGG16 — acc 0.923

	COVID	NORMAL	PNEUMONIA
True COVID	150	3	79
True NORMAL	0	618	16
True PNEUMONIA	5	29	814
	COVID	NORMAL	PNEUMONIA
	Pred		

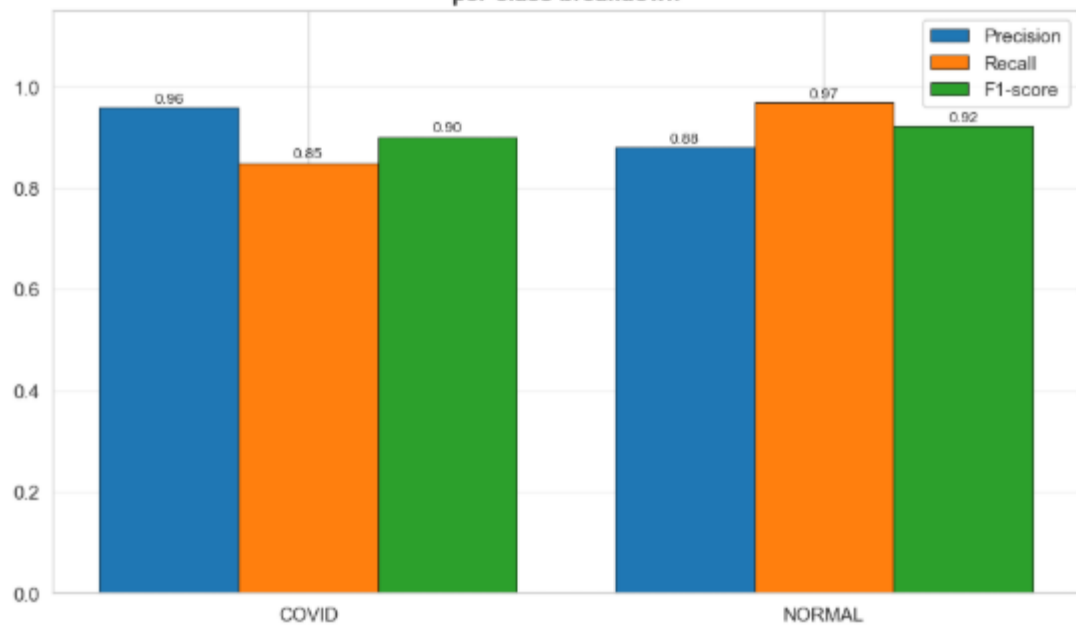


ResNet50 — acc 0.890

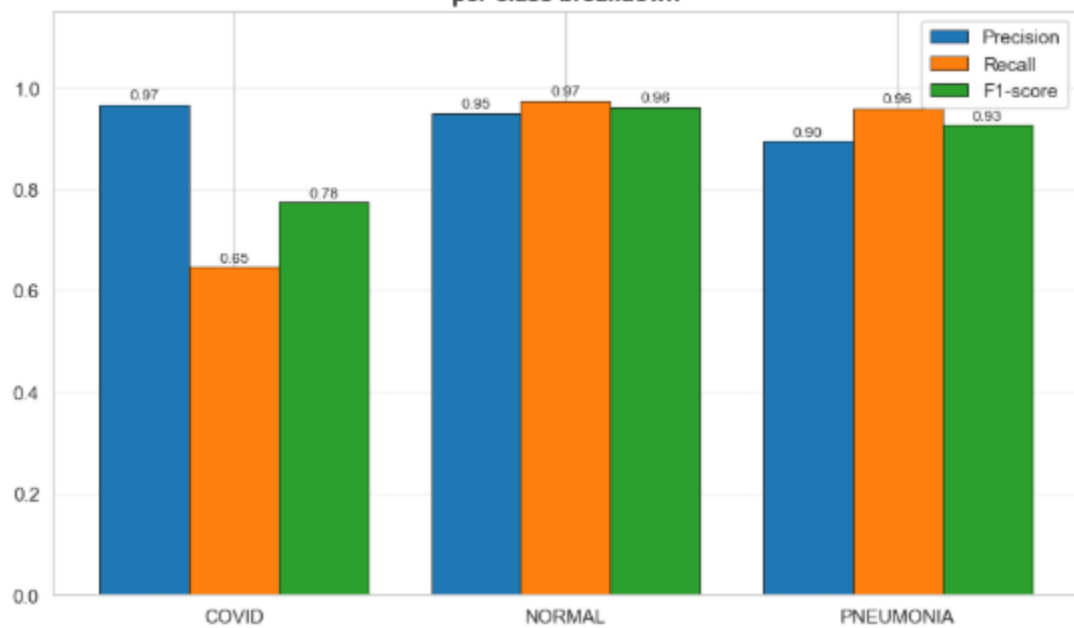
	COVID	NORMAL	PNEUMONIA
True COVID	142	12	78
True NORMAL	4	598	32
True PNEUMONIA	1	61	786
	COVID	NORMAL	PNEUMONIA
	Pred		



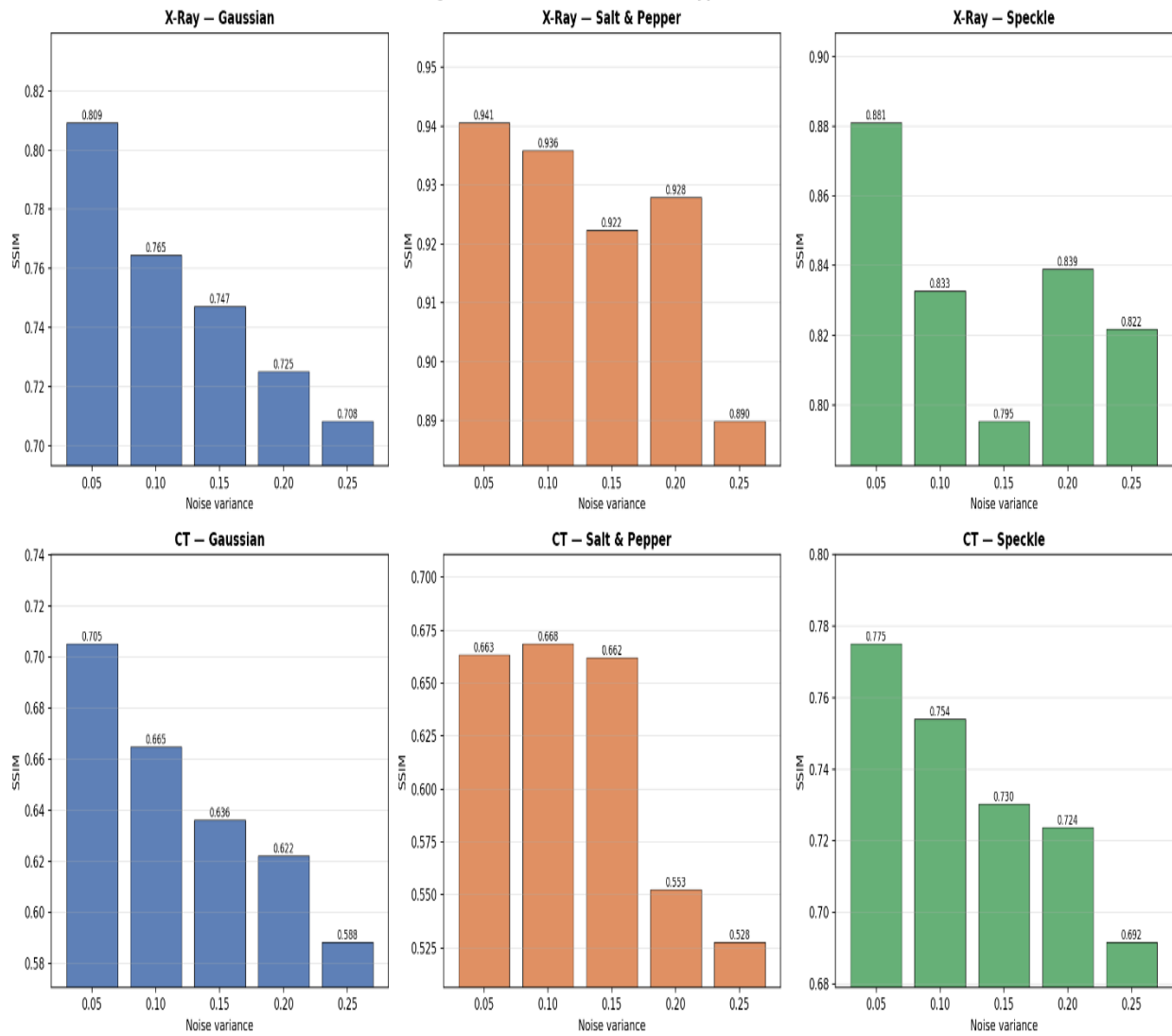
Best on CT scan: VGG16
per-class breakdown

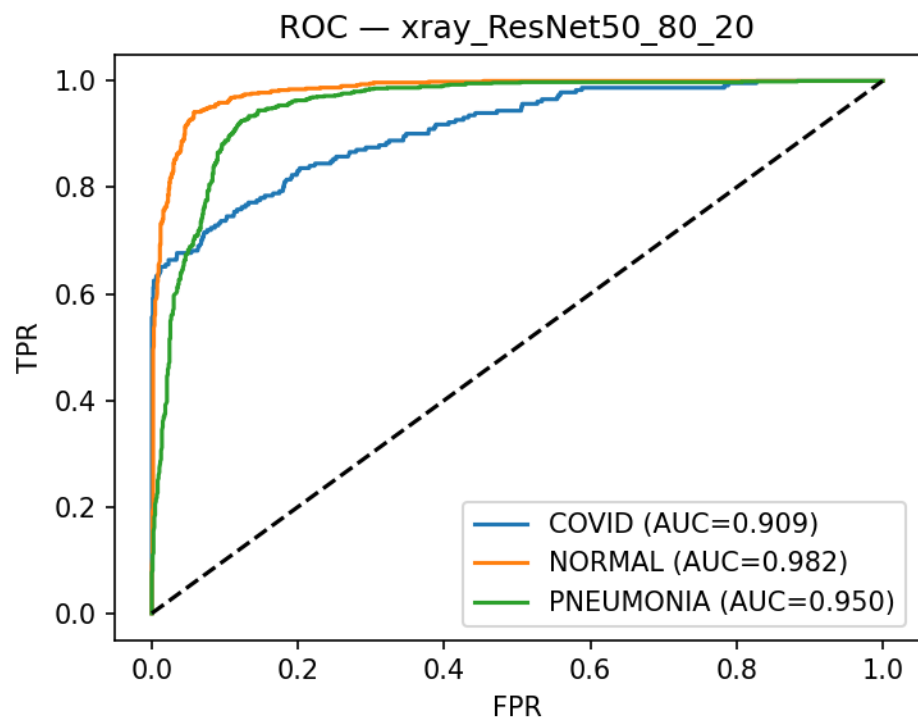
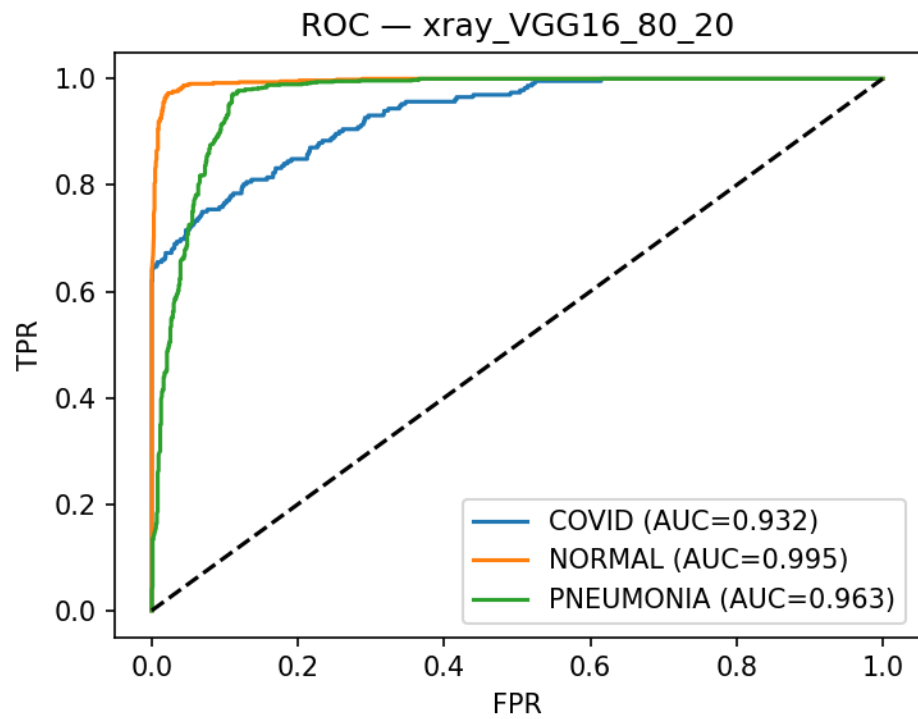


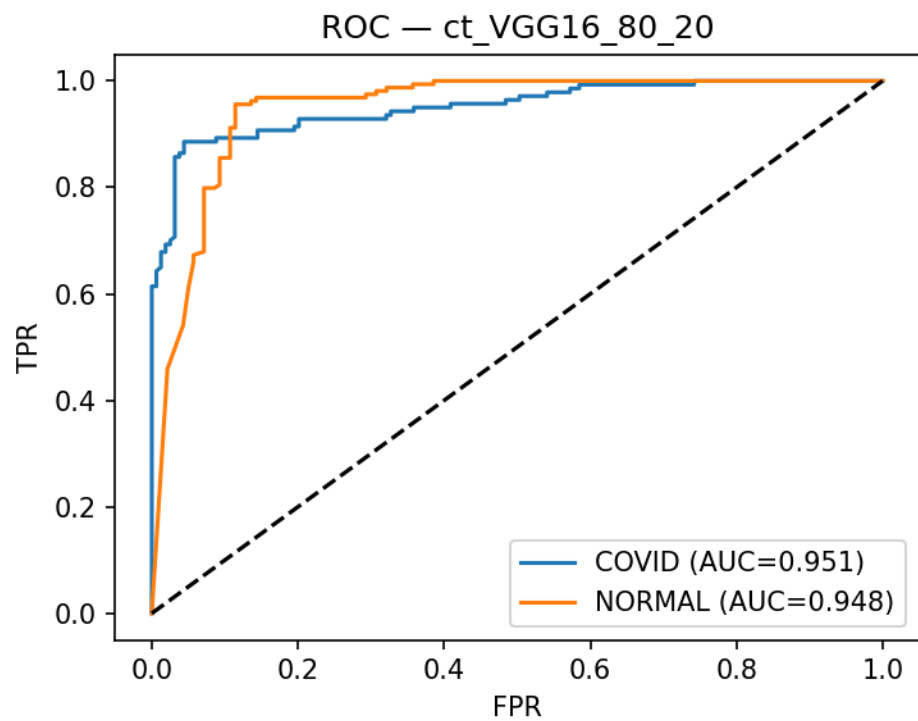
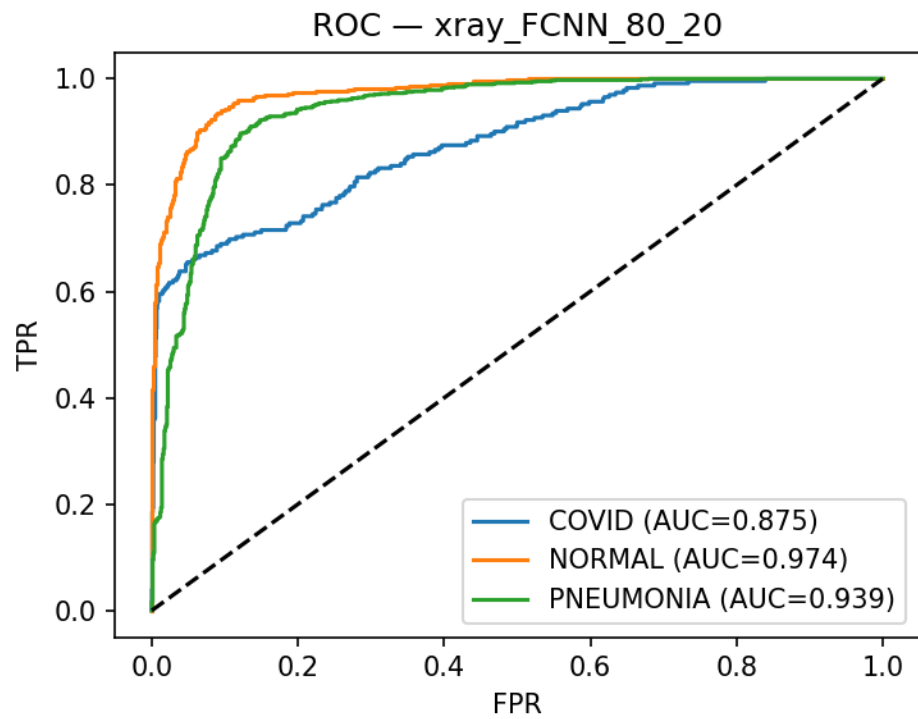
Best on X-Ray: VGG16
per-class breakdown

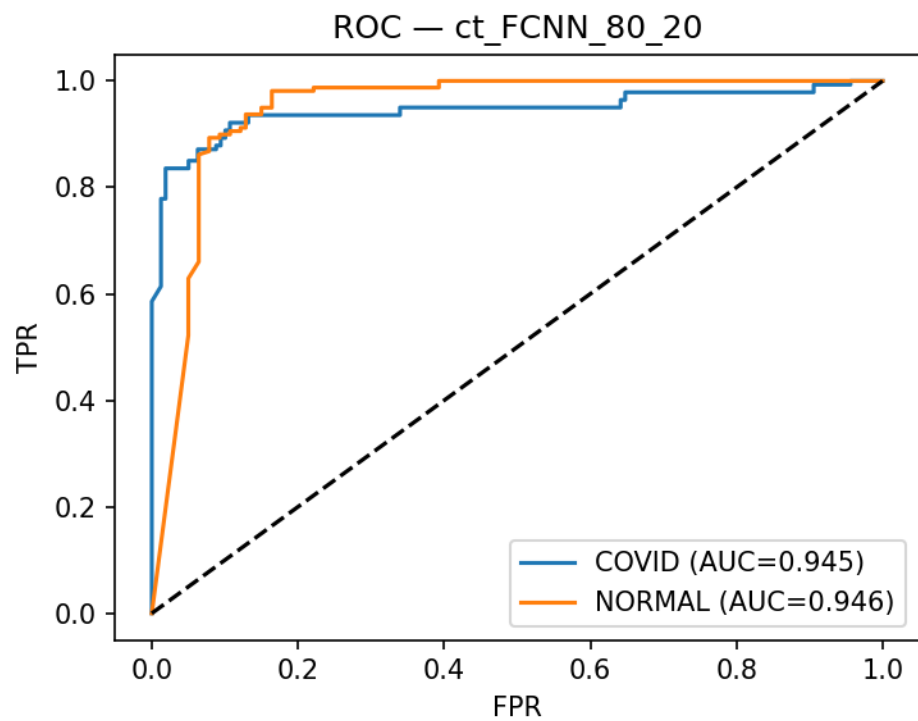
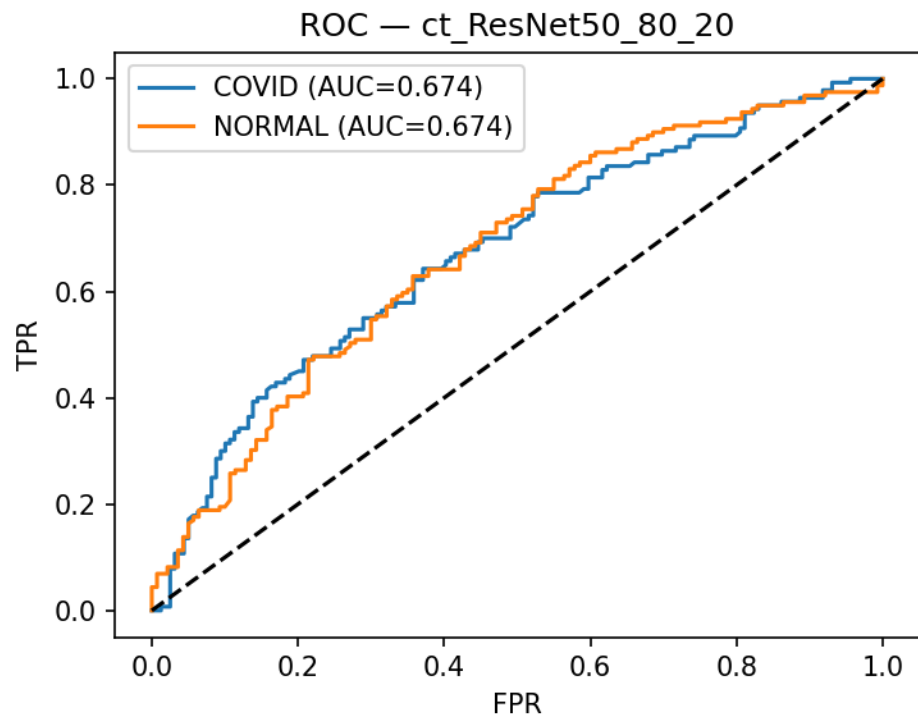


Our Denoising Results – SSIM across noise types and variances

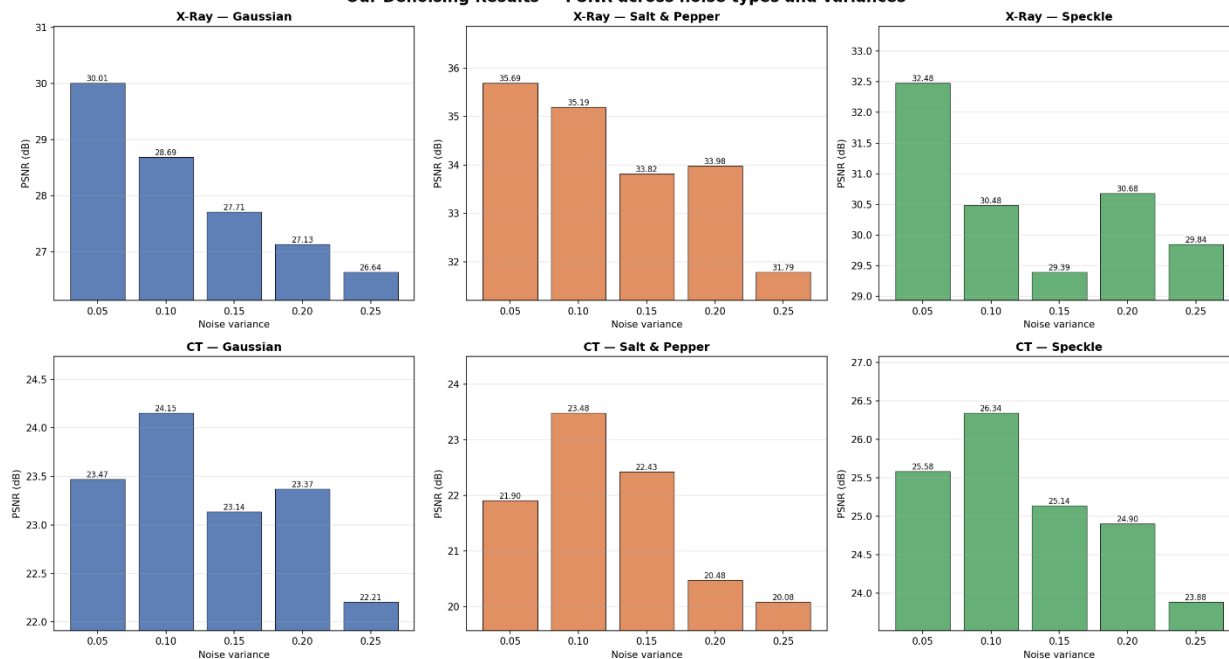




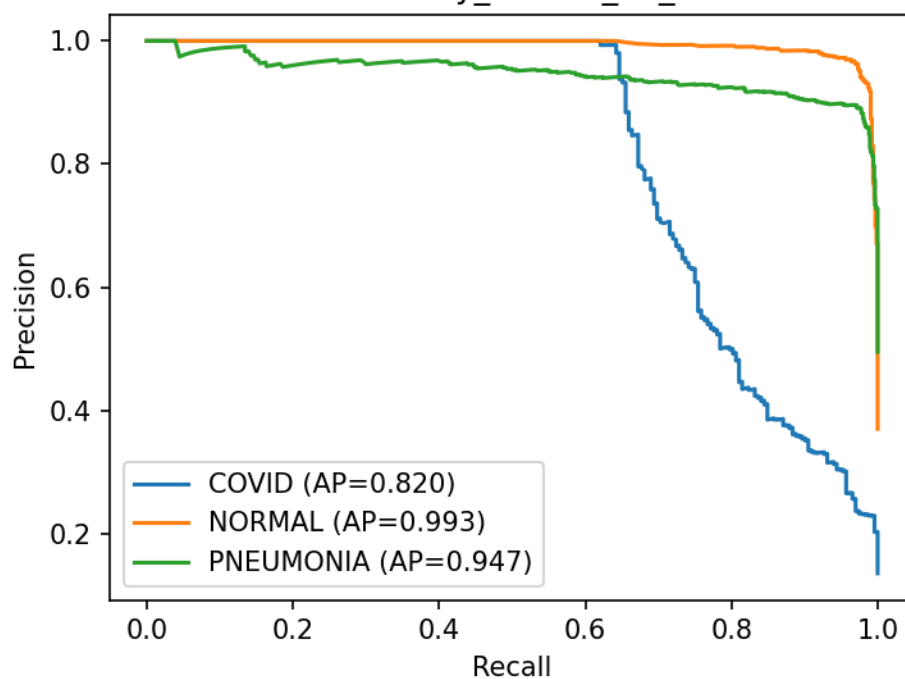


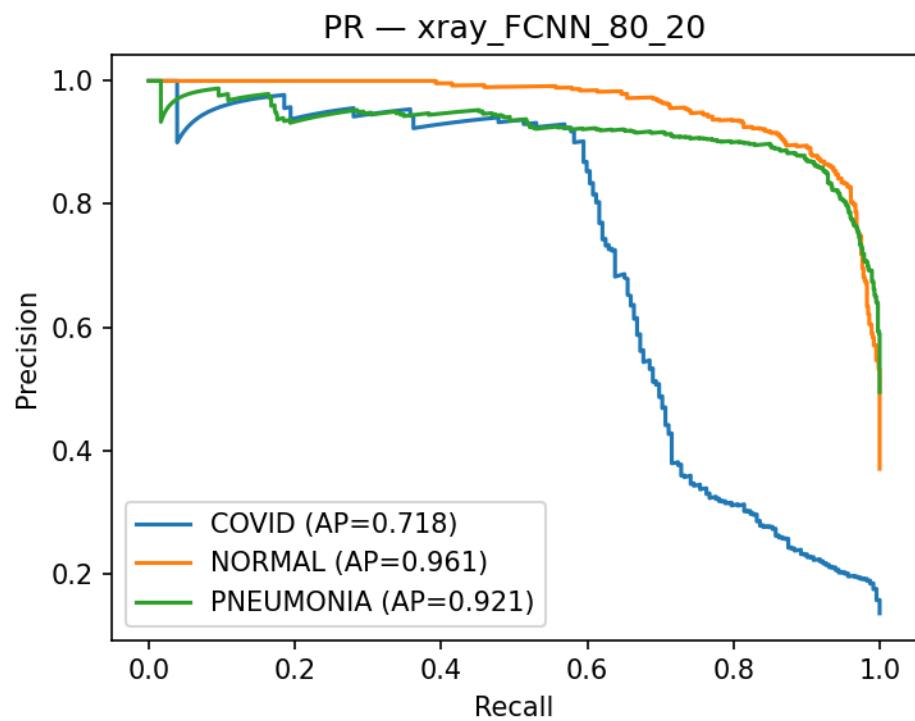
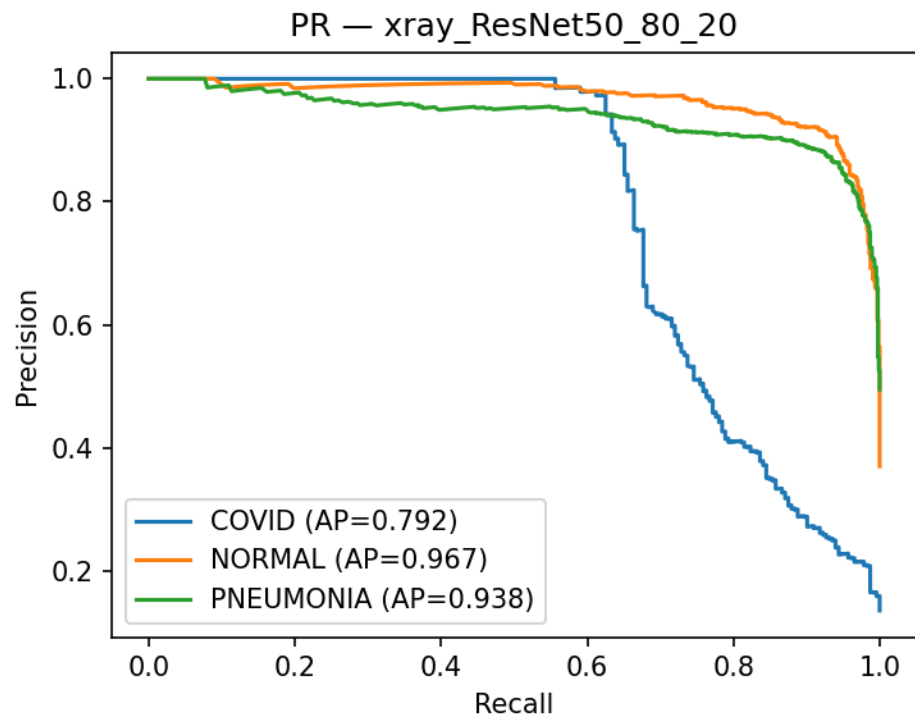


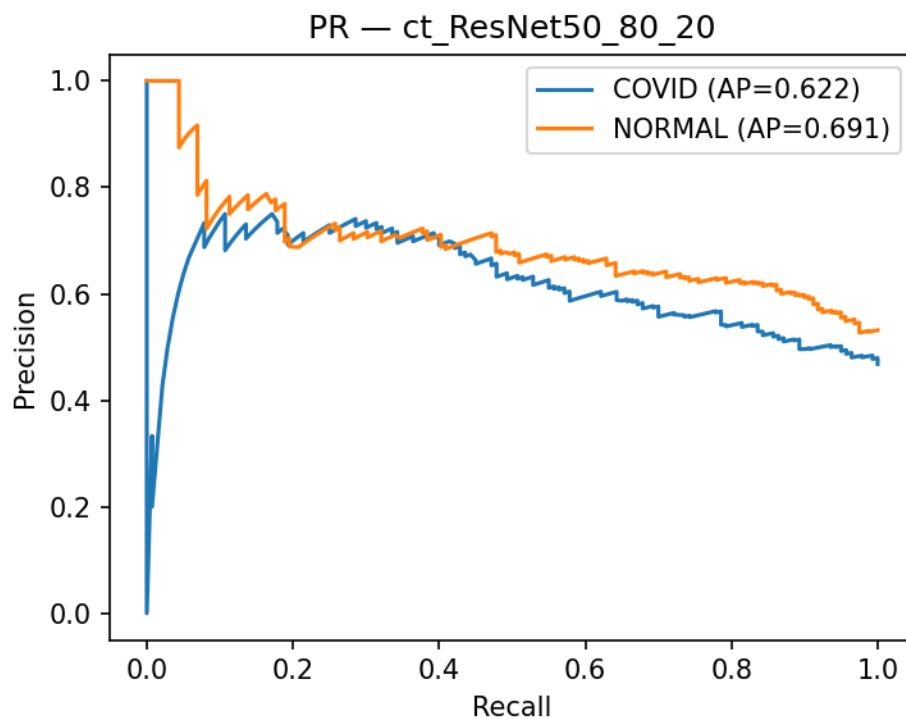
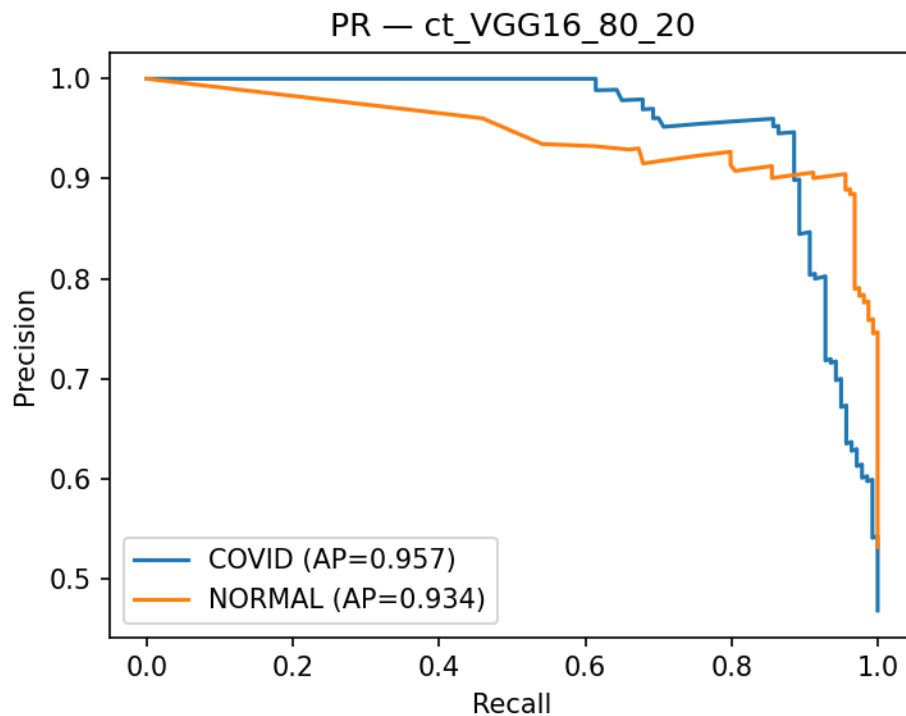
Our Denoising Results — PSNR across noise types and variances

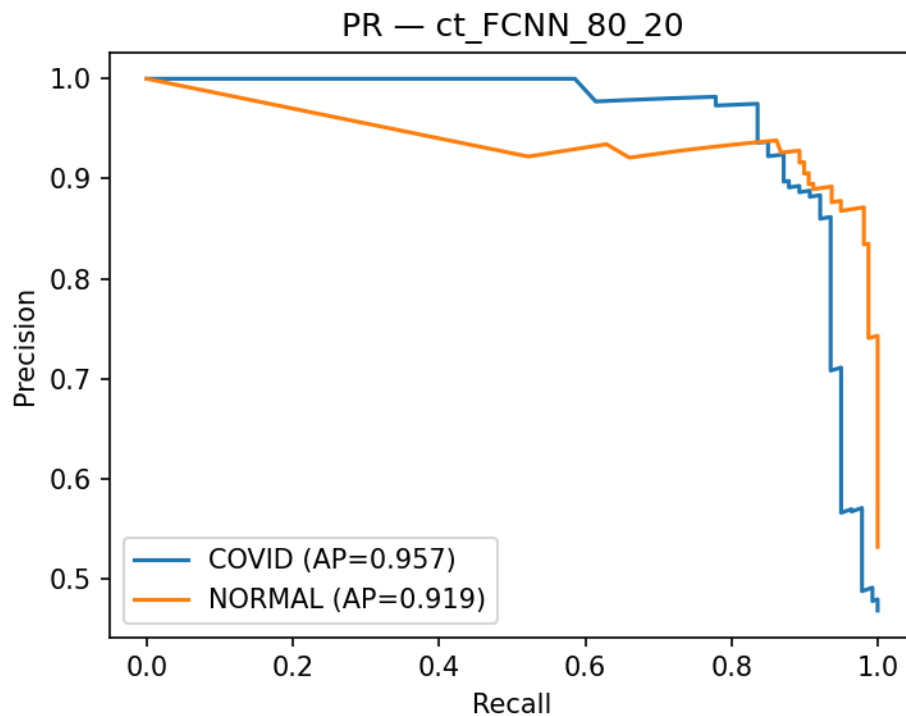


PR — xray_VGG16_80_20

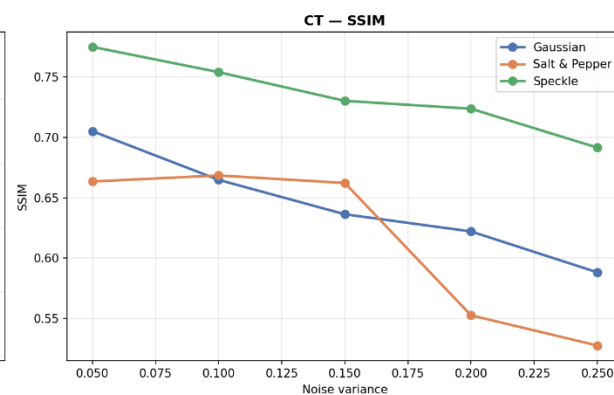
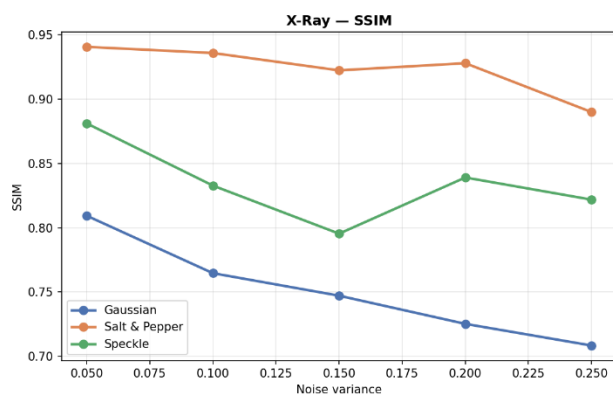
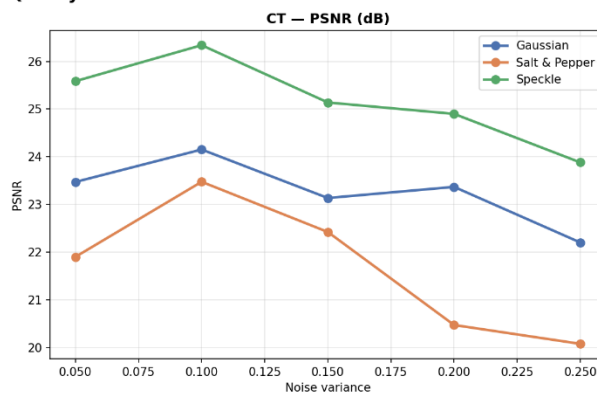
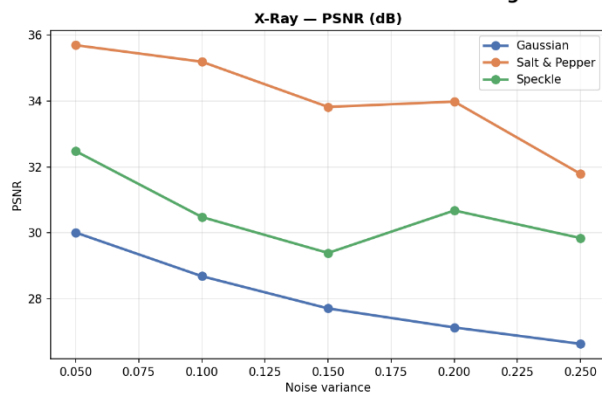




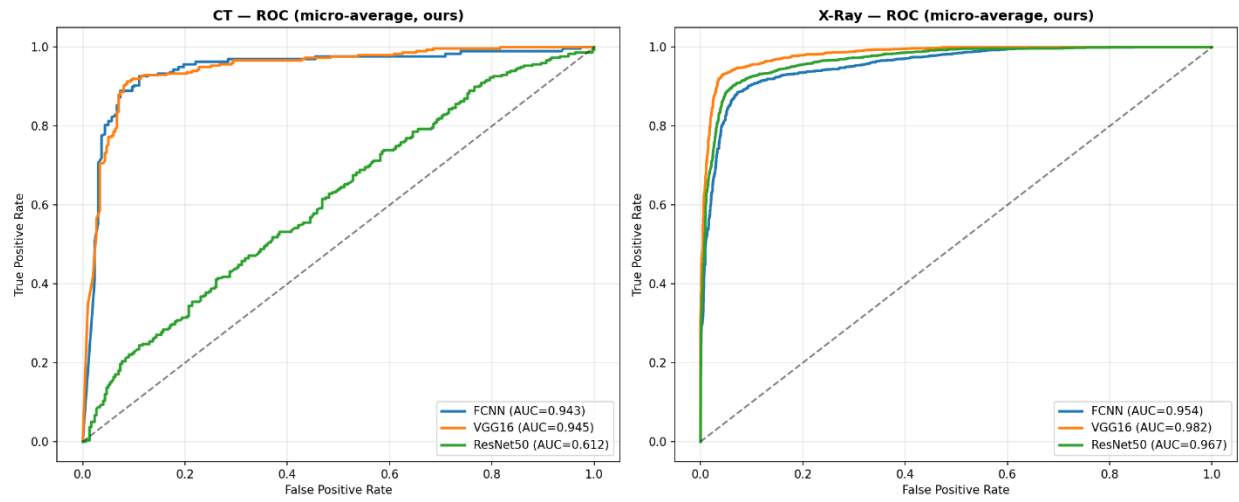




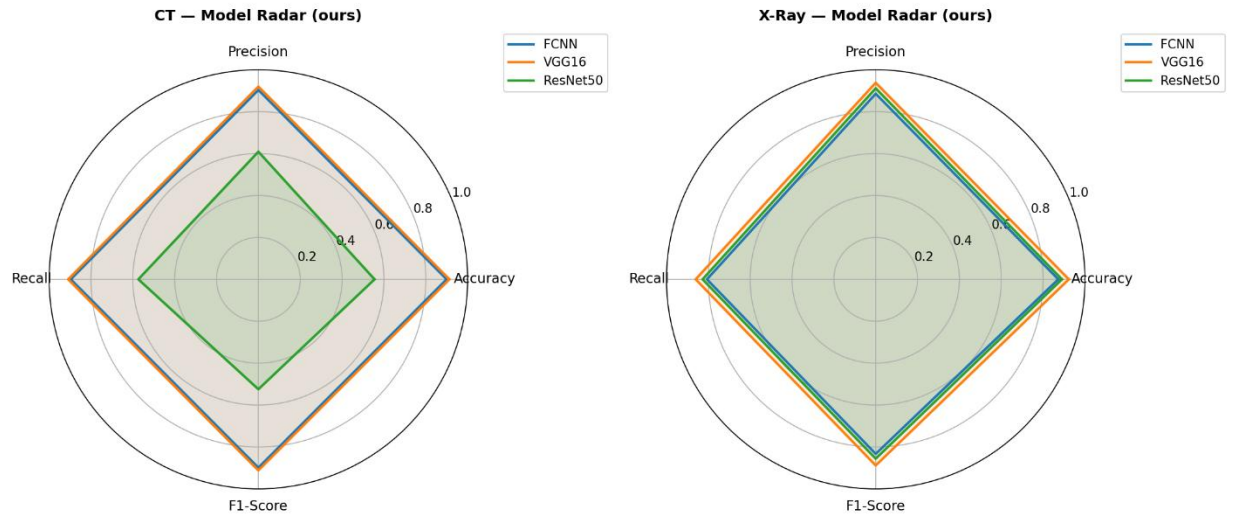
Our Denoising Results — Quality vs Noise Level



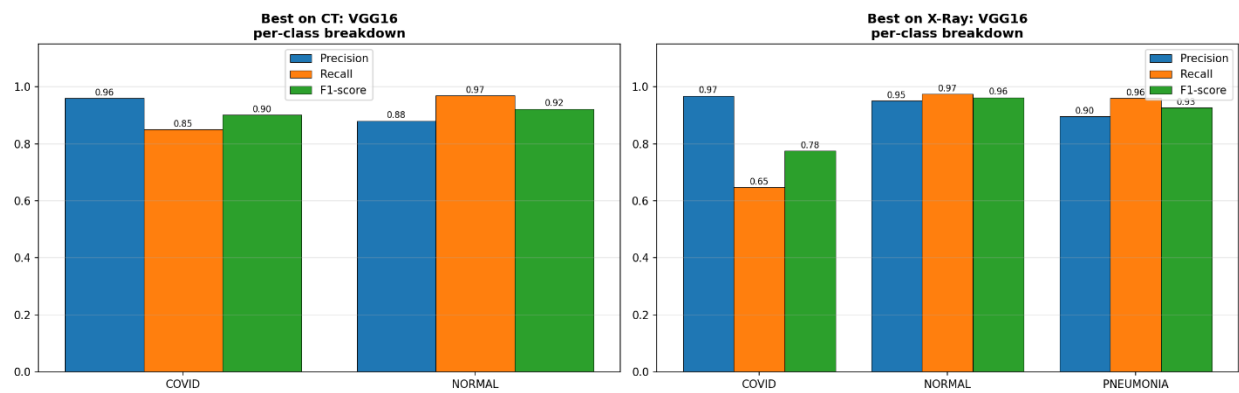
Our Classification Results — ROC Curves



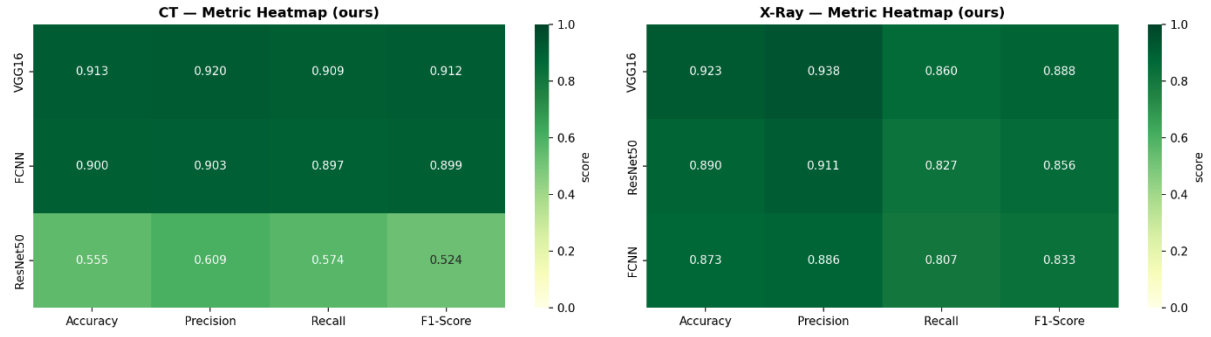
Our Classification Results — Radar Comparison



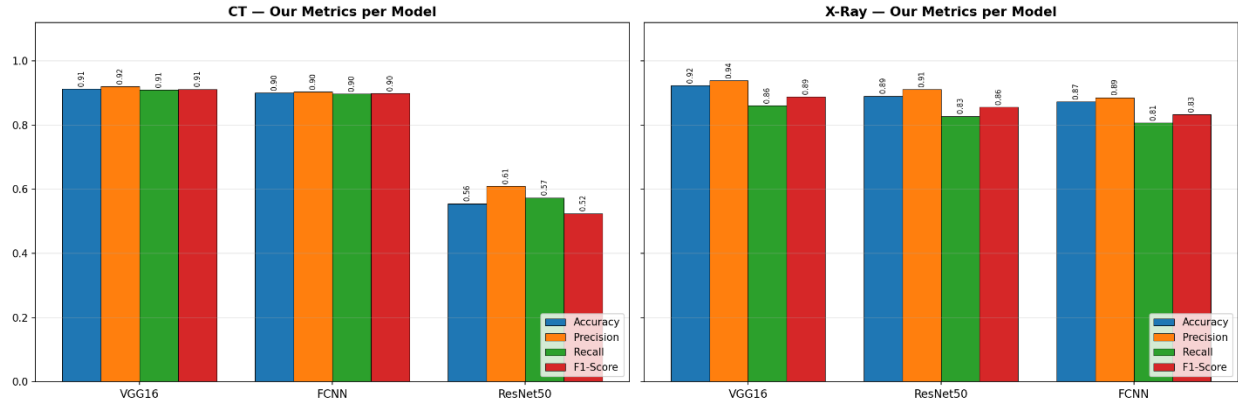
Our Results — Per-Class Metrics of the Best Models



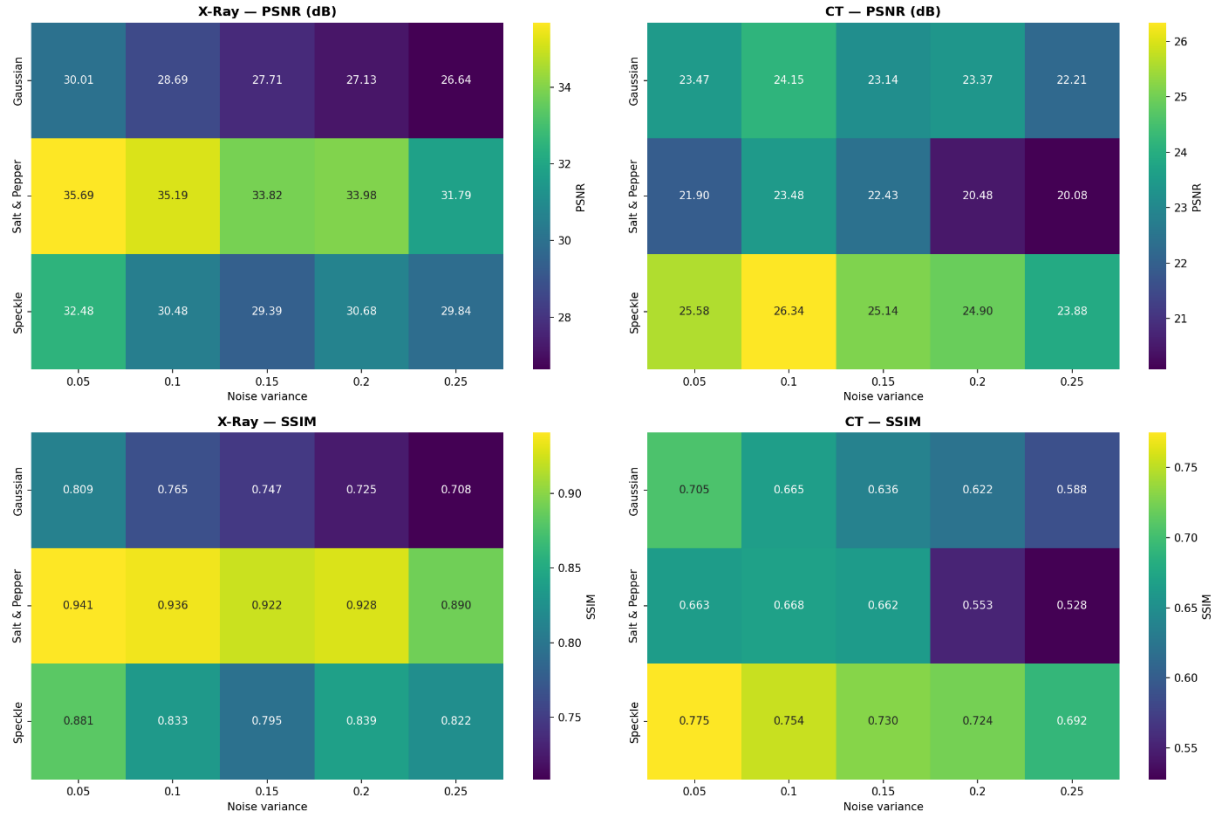
Our Classification Results — Heatmaps



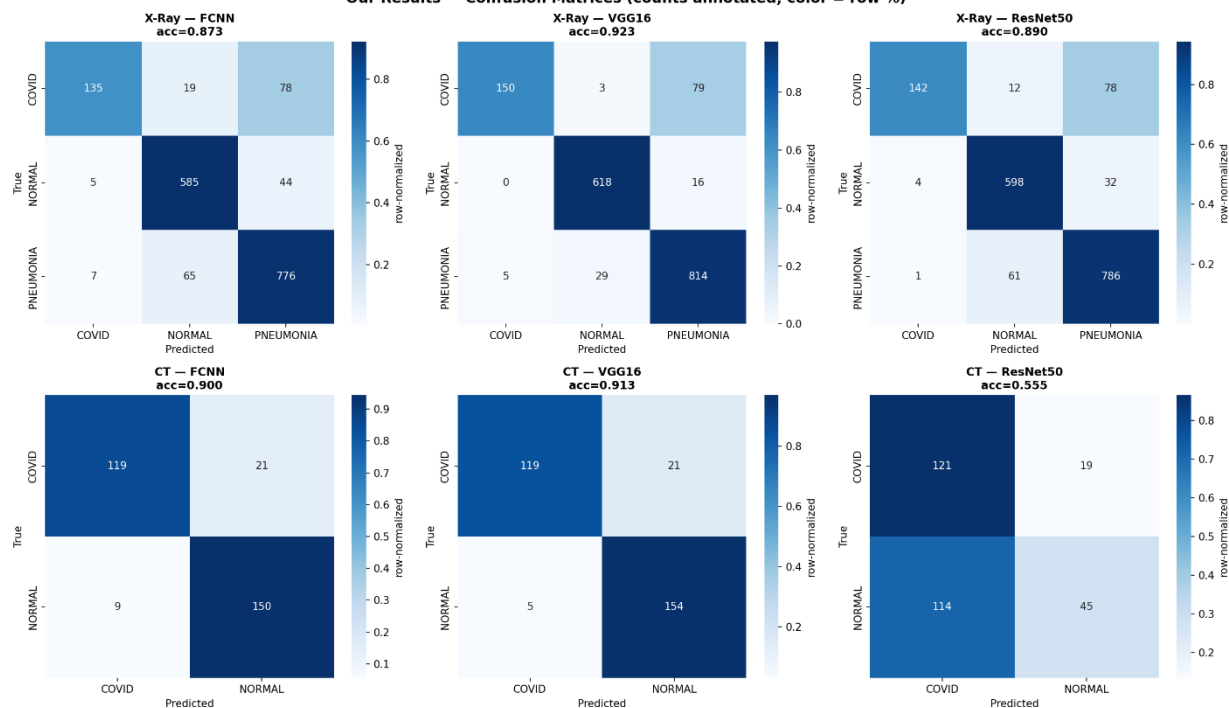
Our Classification Results — All Metrics



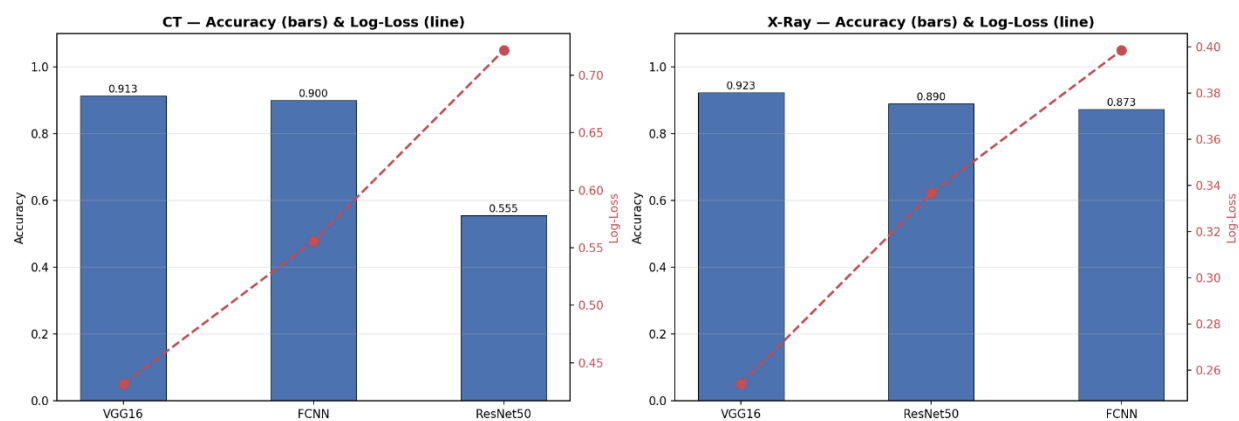
Our Denoising Results — Heatmap Overview



Our Results — Confusion Matrices (counts annotated, color = row %)



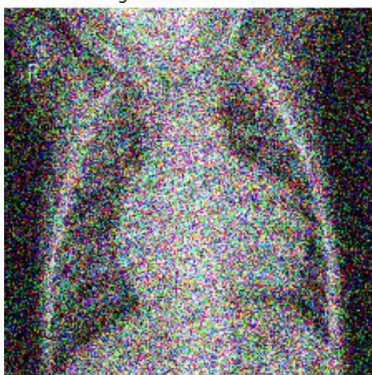
Our Results — Accuracy vs Log-Loss



Clean



gaussian $v=0.15$



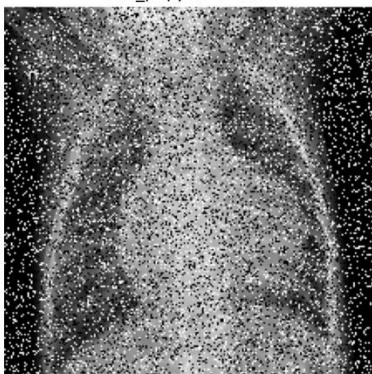
Denoised



Clean



salt_pepper $v=0.15$



Denoised



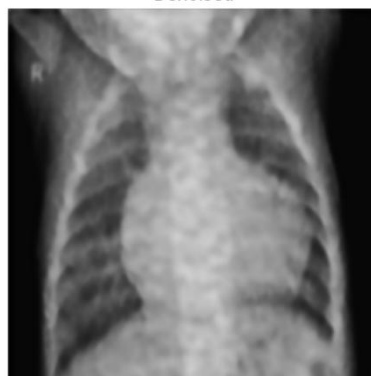
Clean

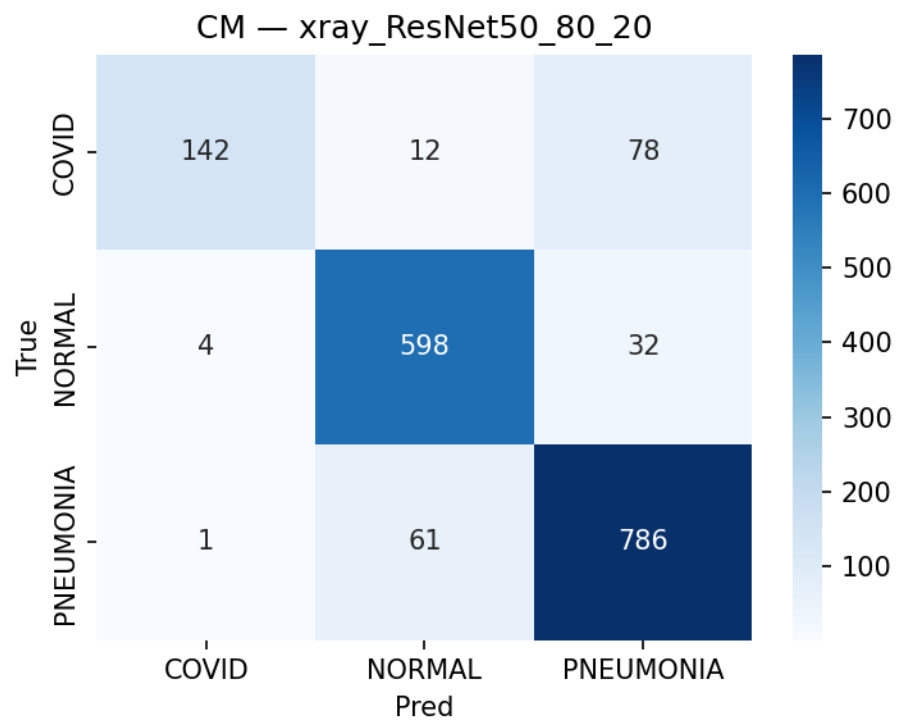
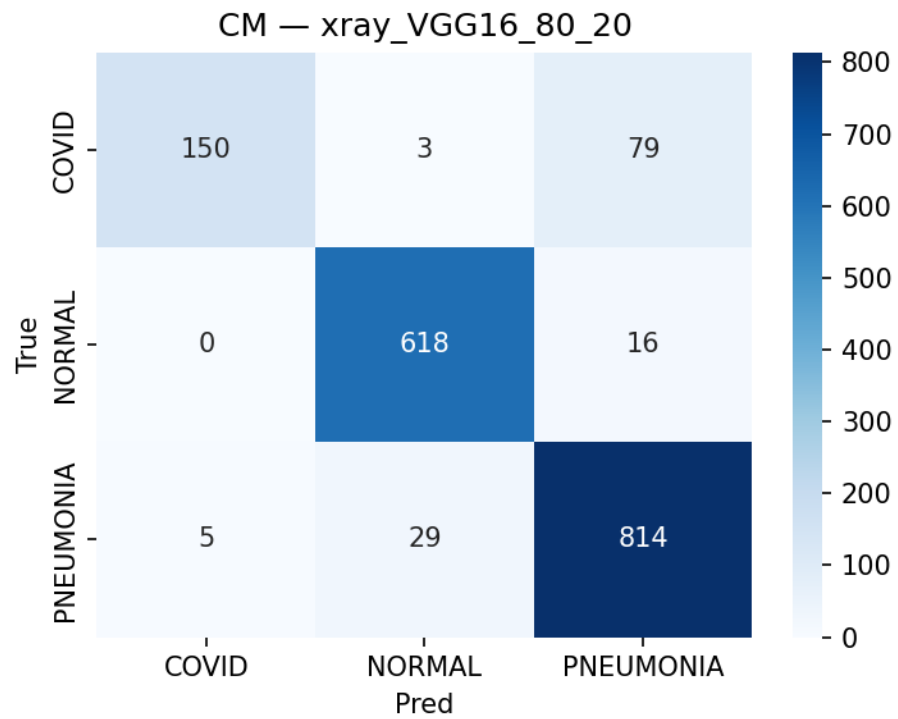


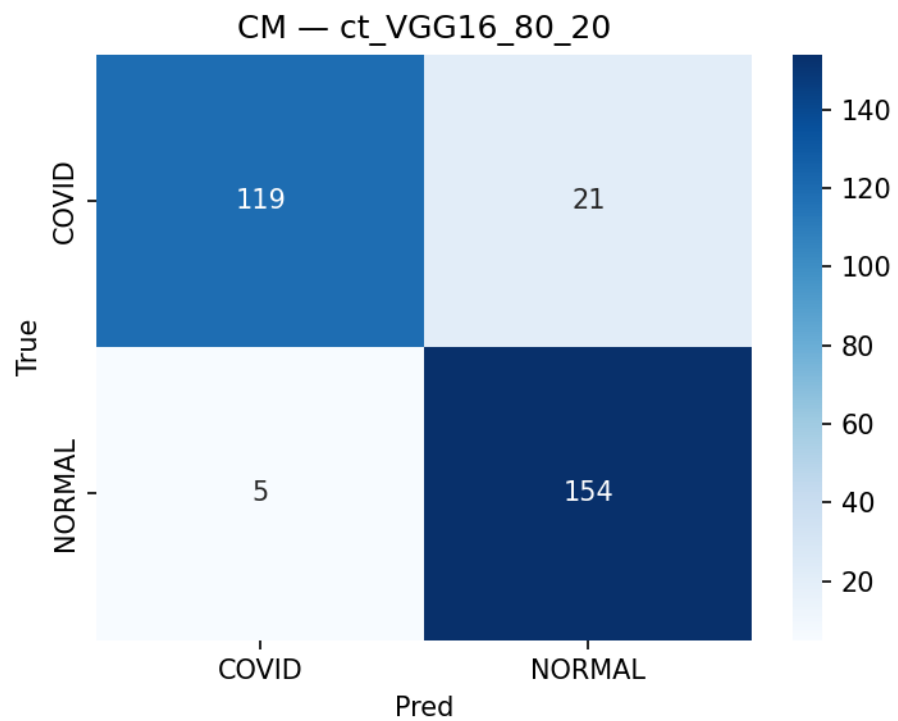
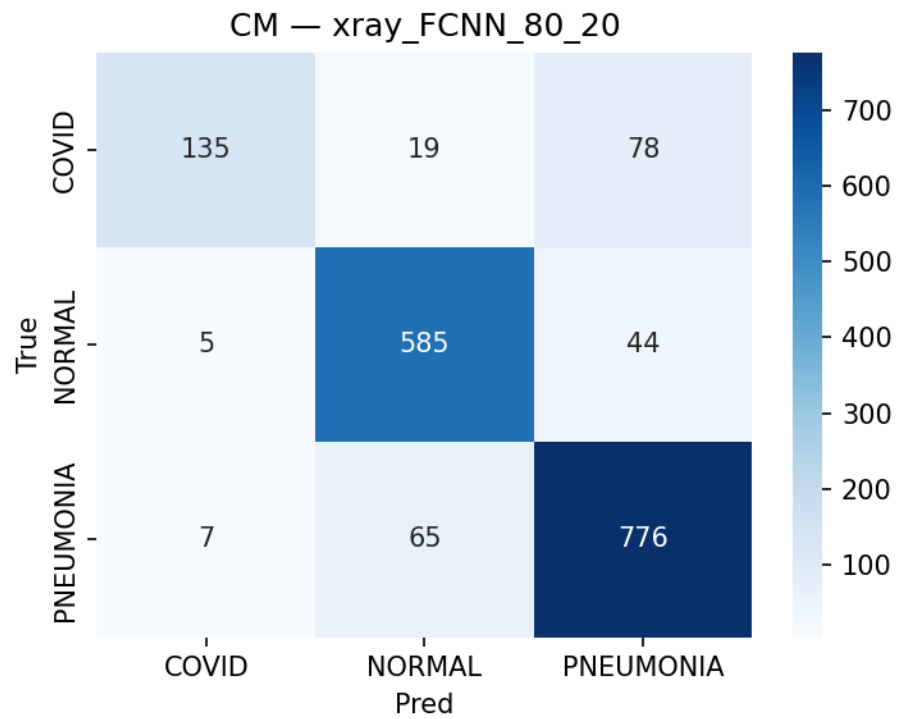
speckle $v=0.15$

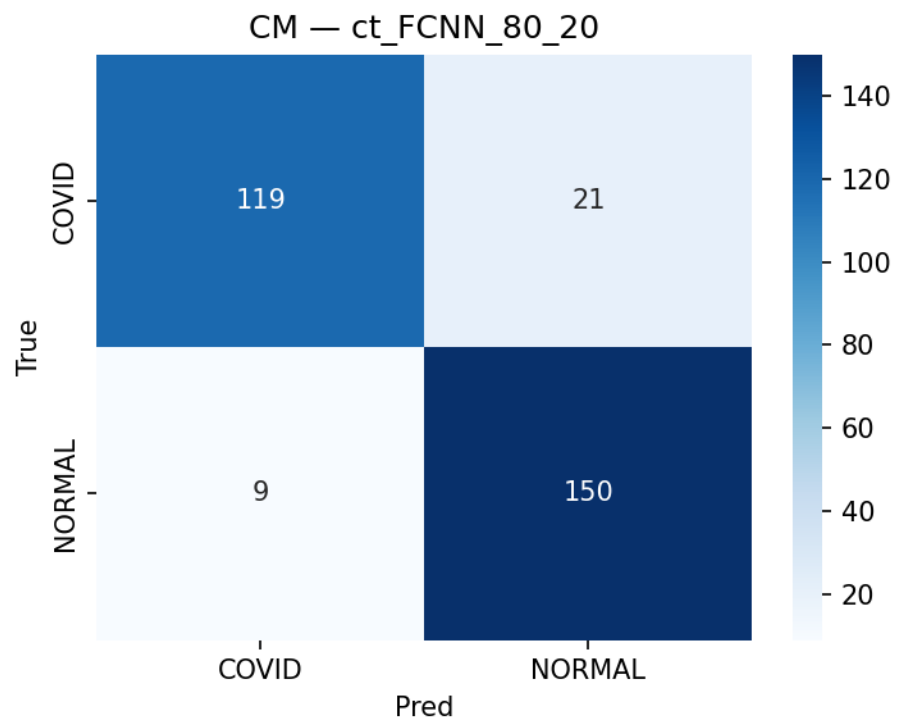
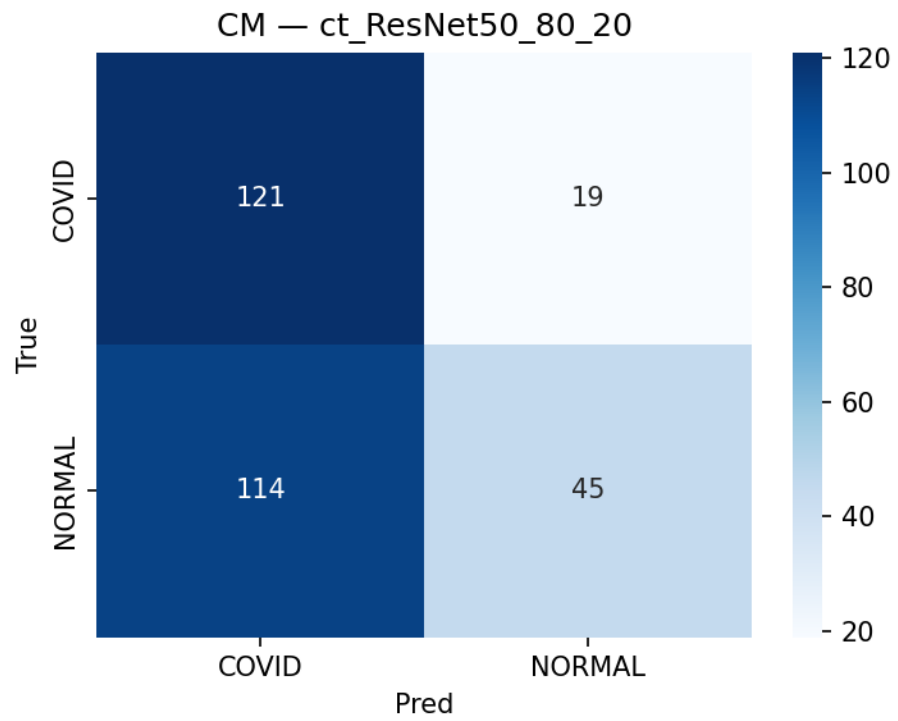


Denoised









ممنون و خسته نباشید.